

ECE/CS 757 Advanced Computer Architecture II

Spring 2025

Consistency (Part 2)

Instructor: Prof. Joshua San Miguel

Some slides adopted from Greg Steffan, Natalie Enright Jerger and Mikko Lipasti

Last Lecture

- SW and HW transactional memory
- TM challenges
- Sequential consistency
- Causality

This Lecture

- Program ordering
- Store atomicity

Sequential Consistency – Program Ordering

Requirements for a sequentially consistent processor:

Program Ordering:

Each individual core's reads and writes must be globally performed in program order.

Store Atomicity:

If a write is performed with respect to core P_i before another write is performed with respect to P_i , the two writes must be performed in the same order with respect to any other core P_j .

- Property that states the existence of a total order of all changes to memory state (i.e., writes)

Program Ordering

Possible: A=1, B=1, C=0, D=0?

```
// init
*flagx = 0; *flagy = 0; *x = 0; *y = 0;
```

```
// P1
*x = 1;
*flagx = 1;
```

```
// P2
A = *flagx;
B = *flagy;
C = *x;
D = *y;
```

```
// P3
*y = 1;
*flagy = 1;
```

Program Ordering

Possible: A=1, B=1, C=0, D=0?

```
// init
*flagx = 0; *flagy = 0; *x = 0; *y = 0;
```

```
// P1
*x = 1;
*flagx = 1;
```

```
// P2
A = *flagx;
B = *flagy;
C = *x;
D = *y;
```

```
// P3
*y = 1;
*flagy = 1;
```

**with program ordering:
other cores must observe all
memory ops in program order**

Program Ordering

Possible: A=1, B=1, C=0, D=0?

```
// init
*flagx = 0; *flagy = 0; *x = 0; *y = 0;
```

```
// P1
*x = 1;
*flagx = 1;
```

with program ordering:

```
*x = 1;
```



```
*flagx = 1;
```

```
// P2
A = *flagx;
B = *flagy;
C = *x;
D = *y;
```

```
// P3
*y = 1;
*flagy = 1;
```

with program ordering:

```
*y = 1;
```



```
*flagy = 1;
```



Program Ordering

Possible: A=1, B=1, C=0, D=0? No

```
// init
```

```
*flagx = 0; *flagy = 0; *x = 0; *y = 0;
```

```
// P1
```

```
*x = 1;
```

```
*flagx = 1;
```

with program ordering:

```
*x = 1;
```

```
↓  
*flagx = 1;
```

WAR

RAW

cycle!

```
// P2
```

```
A = *flagx;
```

```
B = *flagy;
```

```
C = *x;
```

```
D = *y;
```

```
// P3
```

```
*y = 1;
```

```
*flagy = 1;
```

with program ordering:

```
*y = 1;
```

```
↓  
*flagy = 1;
```

WAR

RAW

cycle!



Program Ordering

Possible: A=1, B=1, C=0, D=0?

```
// init
*flagx = 0; *flagy = 0; *x = 0; *y = 0;
```

```
// P1
*x = 1;
*flagx = 1;
```

```
// P2
A = *flagx;
B = *flagy;
C = *x;
D = *y;
```

```
// P3
*y = 1;
*flagy = 1;
```

without program ordering:
other cores may observe
reordering of memory ops

Program Ordering

Possible: A=1, B=1, C=0, D=0?

```
// init
*flagx = 0; *flagy = 0; *x = 0; *y = 0;
```

```
// P1
*x = 1;
*flagx = 1;
```

without program ordering:

```
*flagx = 1;
```



```
*x = 1;
```

```
// P2
A = *flagx;
B = *flagy;
C = *x;
D = *y;
```

```
// P3
*y = 1;
*flagy = 1;
```

without program ordering:

```
*flagy = 1;
```



```
*y = 1;
```



Program Ordering

Possible: A=1, B=1, C=0, D=0? Yes

```
// init
*flagx = 0; *flagy = 0; *x = 0; *y = 0;
```

```
// P1
*x = 1;
*flagx = 1;
```

```
// P2
A = *flagx;
B = *flagy;
C = *x;
D = *y;
```

```
// P3
*y = 1;
*flagy = 1;
```

without program ordering:

```
*flagx = 1;
```

```
*x = 1;
```

RAW

WAR

RAW

WAR

without program ordering:

```
*flagy = 1;
```

```
*y = 1;
```

no cycle



Enforcing Program Ordering

Single-threaded era:

Developed compiler and microarchitecture optimizations that extract ILP

- We realized that not all instructions depend on each other, so we can move them around yet still produce the same final result

Multithreaded era:

Developed *SC-preserving program declarations and compilers*, *load replay* and *memory fences* to maintain per-thread instruction order

- We realized that our instruction reorderings are only valid if we assume that no other entity (e.g., thread) is accessing shared memory state

Program Ordering – Example 1

Possible values: A, C?

```
// init
*x = 0; *y = 0;
```

```
// P1
*x = 8;
A = *y;
```

```
// P2
B = *x * 2;
...
*y = 5;
C = *x * 2;
```

Program Ordering – Example 1

Possible values: A, C?

(A=0 C=16) (A=5 C=0) (A=5 C=16)

```
// init  
*x = 0; *y = 0;
```

```
// P1  
*x = 8;  
A = *y;
```

```
// P2  
B = *x * 2;  
...  
*y = 5;  
C = *x * 2;
```

Program Ordering – Example 1

Possible values: A, C?

(A=0 C=16) (A=5 C=0) (A=5 C=16)

```
// init
*x = 0; *y = 0;
```

```
// P1
*x = 8;
A = *y;
```

```
// P2
B = *x * 2;
```

...

```
*y = 5;
```

```
C = *x * 2;
```

common subexpression elimination



Program Ordering – Example 1

Possible values: A, C?

(A=0 C=16) (A=5 C=0) (A=5 C=16)

```
// init
*x = 0; *y = 0;
```

```
// P1
*x = 8;
A = *y;
```

```
// P2
B = *x * 2;
```

...

```
*y = 5;
```

```
C = B;
```

common subexpression elimination



Program Ordering – Example 1

Possible values: A, C?

(A=0 C=16) (A=5 C=0) (A=5 C=16)

```
// init
*x = 0; *y = 0;
```

```
// P1
*x = 8;
A = *y;
```

```
// P2
B = *x * 2;
...
*y = 5;
C = B;
```

Program Ordering – Example 1

Possible values: A, C?

(A=0 C=16) (A=5 C=0) (A=5 C=16) (**A=0 C=0**)

SC violation!

```
// init
*x = 0; *y = 0;
```

```
// P1
*x = 8;
A = *y;
```

```
// P2
B = *x * 2;
...
*y = 5;
C = B;
```



Program Ordering – Example 2

Possible assertion fail?

```
// init
*x = 0; *y = 0;
```

```
// P1
*x = 7;
*y = 9;
```

```
// P2
if(*y != 0) {
    A = *x;
    assert(A != 0);
} else {
    A = *x;
    ...
}
```

Program Ordering – Example 2

Possible assertion fail?

No

```
// init
*x = 0; *y = 0;
```

```
// P1
*x = 7;
*y = 9;
```

```
// P2
if(*y != 0) {
    A = *x;
    assert(A != 0);
} else {
    A = *x;
    ...
}
```

Program Ordering – Example 2

Possible assertion fail?

No

```
// init
*x = 0; *y = 0;
```

```
// P1
*x = 7;
*y = 9;
```

```
// P2
if(*y != 0) {
    A = *x;
    assert(A != 0);
} else {
    A = *x;
    ...
}
```

code hoisting

Program Ordering – Example 2

Possible assertion fail?

No

```
// init
*x = 0; *y = 0;
```

```
// P1
*x = 7;
*y = 9;
```

```
// P2
A = *x;
if(*y != 0) {
    assert(A != 0);
} else {
    ...
}
```

code hoisting



Program Ordering – Example 2

Possible assertion fail?

No

```
// init
*x = 0; *y = 0;
```

```
// P1
*x = 7;
*y = 9;
```

```
// P2
A = *x;
if(*y != 0) {
    assert(A != 0);
} else {
    ...
}
```

Program Ordering – Example 2

Possible assertion fail?

Yes

SC violation!

```
// init
*x = 0; *y = 0;
```

```
// P1
*x = 7;
*y = 9;
```

```
// P2
A = *x;
if(*y != 0) {
    assert(A != 0);
} else {
    ...
}
```



Program Ordering – Example 3

Possible assertion fail?

No

```
// init
*x = 0; *y = 0;
```

```
// P1
*x = 7;
*y = 9;
```

```
// P2
if(*y != 0) {
    A = *x;
    assert(A != 0);
} else {
    A = *x;
    ...
}
```

Program Ordering – Example 3

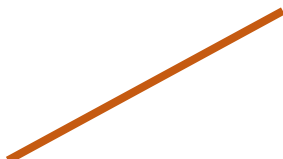
Possible assertion fail?

No

```
// init
*x = 0; *y = 0;
```

```
// P1
*x = 7;
*y = 9;
```

independent instructions
according to compiler



```
// P2
if(*y != 0) {
    A = *x;
    assert(A != 0);
} else {
    A = *x;
    ...
}
```



Program Ordering – Example 3

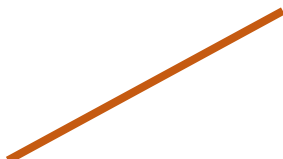
Possible assertion fail?

No

```
// init
*x = 0; *y = 0;
```

```
// P1
*y = 9;
*x = 7;
```

independent instructions
according to compiler



```
// P2
if(*y != 0) {
    A = *x;
    assert(A != 0);
} else {
    A = *x;
    ...
}
```



Program Ordering – Example 3

Possible assertion fail?

No

```
// init
*x = 0; *y = 0;
```

```
// P1
*y = 9;
*x = 7;
```

```
// P2
if(*y != 0) {
    A = *x;
    assert(A != 0);
} else {
    A = *x;
    ...
}
```

Program Ordering – Example 3

Possible assertion fail?

Yes

SC violation!

```
// init
*x = 0; *y = 0;
```

```
// P1
*y = 9;
*x = 7;
```

```
// P2
if(*y != 0) {
    A = *x;
    assert(A != 0);
} else {
    A = *x;
    ...
}
```

Program Ordering – Example 4

Possible result: A=B=C=2?

```
// init  
A = 0; B = 0; *x = 1; *y = 0;
```

```
// P1  
A = *x;  
B = *x;  
if(A == B)  
    *y = 2;
```

```
// P2  
C = *y;  
*x = C;
```

Program Ordering – Example 4

Possible result: A=B=C=2?

No

```
// init  
A = 0; B = 0; *x = 1; *y = 0;
```

```
// P1  
A = *x;  
B = *x;  
if(A == B)  
    *y = 2;
```

```
// P2  
C = *y;  
*x = C;
```

Program Ordering – Example 4

Possible result: $A=B=C=2$?

No

```
// init
```

```
A = 0; B = 0; *x = 1; *y = 0;
```

```
// P1
```

```
A = *x;
```

```
B = *x;
```

```
if(A == B)
```

```
    *y = 2;
```

```
// P2
```

```
C = *y;
```

```
*x = C;
```

common subexpression elimination



Program Ordering – Example 4

Possible result: $A=B=C=2$?

No

```
// init
```

```
A = 0; B = 0; *x = 1; *y = 0;
```

```
// P1
```

```
A = *x;
```

```
B = A;
```

```
if(A == B)
```

```
    *y = 2;
```

```
// P2
```

```
C = *y;
```

```
*x = C;
```

common subexpression elimination



Program Ordering – Example 4

Possible result: $A=B=C=2$?

No

```
// init  
A = 0; B = 0; *x = 1; *y = 0;
```

```
// P1  
A = *x;  
B = A;  
if(A == B)  
    *y = 2;
```

```
// P2  
C = *y;  
*x = C;
```

static true

Program Ordering – Example 4

Possible result: A=B=C=2?

No

```
// init  
A = 0; B = 0; *x = 1; *y = 0;
```

```
// P1  
A = *x;  
B = A;  
*y = 2;
```

```
// P2  
C = *y;  
*x = C;
```

Program Ordering – Example 4

Possible result: A=B=C=2?

No

```
// init  
A = 0; B = 0; *x = 1; *y = 0;
```

```
// P1  
A = *x;  
B = A;  
*y = 2;
```

```
// P2  
C = *y;  
*x = C;
```

independent

Program Ordering – Example 4

Possible result: A=B=C=2?

No

```
// init  
A = 0; B = 0; *x = 1; *y = 0;
```

// P1		// P2
*y = 2;		C = *y;
A = *x;		*x = C;
B = A;		

independent

Program Ordering – Example 4

Possible result: A=B=C=2?

No

```
// init  
A = 0; B = 0; *x = 1; *y = 0;
```

```
// P1  
*y = 2;  
A = *x;  
B = A;
```

```
// P2  
C = *y;  
*x = C;
```

Program Ordering – Example 4

Possible result: $A=B=C=2$?

Yes

SC violation!

```
// init  
A = 0; B = 0; *x = 1; *y = 0;
```

```
// P1  
*y = 2;  
A = *x;  
B = A;
```

```
// P2  
C = *y;  
*x = C;
```

Enforcing Program Ordering – SC Compilers

SC-preserving compilers restrict optimizations (e.g., common subexpression elimination) so that they never reorder memory accesses

- [Lattner and Adve, CGO 2004]
- Simple, small modifications to base compiler
- Average 3.8% slowdown for LLVM on x86

Enforcing Program Ordering – Volatile

Language declaration that allows the programmer to tell the compiler that an external entity (e.g., another thread, I/O device) may modify the memory location, so must be conservative and limit optimizations.

Enforcing Program Ordering – Volatile

Language declaration that allows the programmer to tell the compiler that an external entity (e.g., another thread, I/O device) may modify the memory location, so must be conservative and limit optimizations.

```
int *x = ...;
```

```
*x = A;
```

```
...
```

```
B = *x;
```

Enforcing Program Ordering – Volatile

Language declaration that allows the programmer to tell the compiler that an external entity (e.g., another thread, I/O device) may modify the memory location, so must be conservative and limit optimizations.

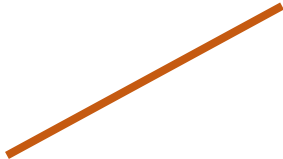
```
int *x = ...;
```

```
*x = A;
```

```
...
```

```
B = *x;
```

compiler may optimize to reuse register value,
since it assumes that the value of *x is unchanged



Enforcing Program Ordering – Volatile

Language declaration that allows the programmer to tell the compiler that an external entity (e.g., another thread, I/O device) may modify the memory location, so must be conservative and limit optimizations.

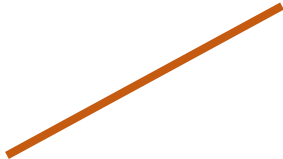
```
int *x = ...;
```

```
*x = A;
```

```
...
```

```
B = A;
```

compiler may optimize to reuse register value,
since it assumes that the value of *x is unchanged



Enforcing Program Ordering – Volatile

Language declaration that allows the programmer to tell the compiler that an external entity (e.g., another thread, I/O device) may modify the memory location, so must be conservative and limit optimizations.

```
int *x = ...;
```

```
*x = A;
```

```
...
```

```
B = *x;
```

Enforcing Program Ordering – Volatile

Language declaration that allows the programmer to tell the compiler that an external entity (e.g., another thread, I/O device) may modify the memory location, so must be conservative and limit optimizations.

```
volatile int *x = ...;
```

```
*x = A;
```

```
...
```

```
B = *x;
```

Enforcing Program Ordering – Volatile

Language declaration that allows the programmer to tell the compiler that an external entity (e.g., another thread, I/O device) may modify the memory location, so must be conservative and limit optimizations.

```
volatile int *x = ...;
```

```
*x = A;
```

```
...
```

```
B = *x;
```

compiler must assume that *x can be
modified by someone else anytime

Enforcing Program Ordering – Example

Possible assertion fail?

```
// init
*x = 0; *y = 0;
```

```
// P1
*x = 7;
*y = 9;
```

```
// P2
if(*y != 0) {
    A = *x;
    assert(A != 0);
} else {
    A = *x;
    ...
}
```

Enforcing Program Ordering – Example

Possible assertion fail?

compiler must be conservative
and preserve ordering

```
// init
volatile *x = 0; volatile *y = 0;

// P1
*x = 7;
*y = 9;

// P2
if(*y != 0) {
    A = *x;
    assert(A != 0);
} else {
    A = *x;
    ...
}
```

Enforcing Program Ordering – Example

Possible assertion fail?

No

compiler must be conservative
and preserve ordering

```
// init
volatile *x = 0; volatile *y = 0;

// P1
*x = 7;
*y = 9;

// P2
if(*y != 0) {
    A = *x;
    assert(A != 0);
} else {
    A = *x;
    ...
}
```

Program Ordering – Example 5

Possible values: A, B?

```
// init
*x = 0; *y = 0;
```

```
// P1
*x = 1;
A = *y;
```

```
// P2
*y = 1;
B = *x;
```

Program Ordering – Example 5

Possible values: A, B?

(A=0 B=1) (A=1 B=0) (A=1 B=1)

```
// init
*x = 0; *y = 0;
```

```
// P1
*x = 1;
A = *y;
```

```
// P2
*y = 1;
B = *x;
```

Program Ordering – Example 5

Possible values: A, B?

(A=0 B=1) (A=1 B=0) (A=1 B=1)

```
// init  
*x = 0; *y = 0;
```

```
// P1  
*x = 1;  
A = *y;
```

```
// P2  
*y = 1;  
B = *x;
```

possible if out-of-order execution
with speculative loads

Program Ordering – Example 5

Possible values: A, B?

(A=0 B=1) (A=1 B=0) (A=1 B=1)

```
// init
*x = 0; *y = 0;
```

```
// P1
*x = 1;
A = *y;
```

← (1) GetS y

```
// P2
*y = 1;
B = *x;
```

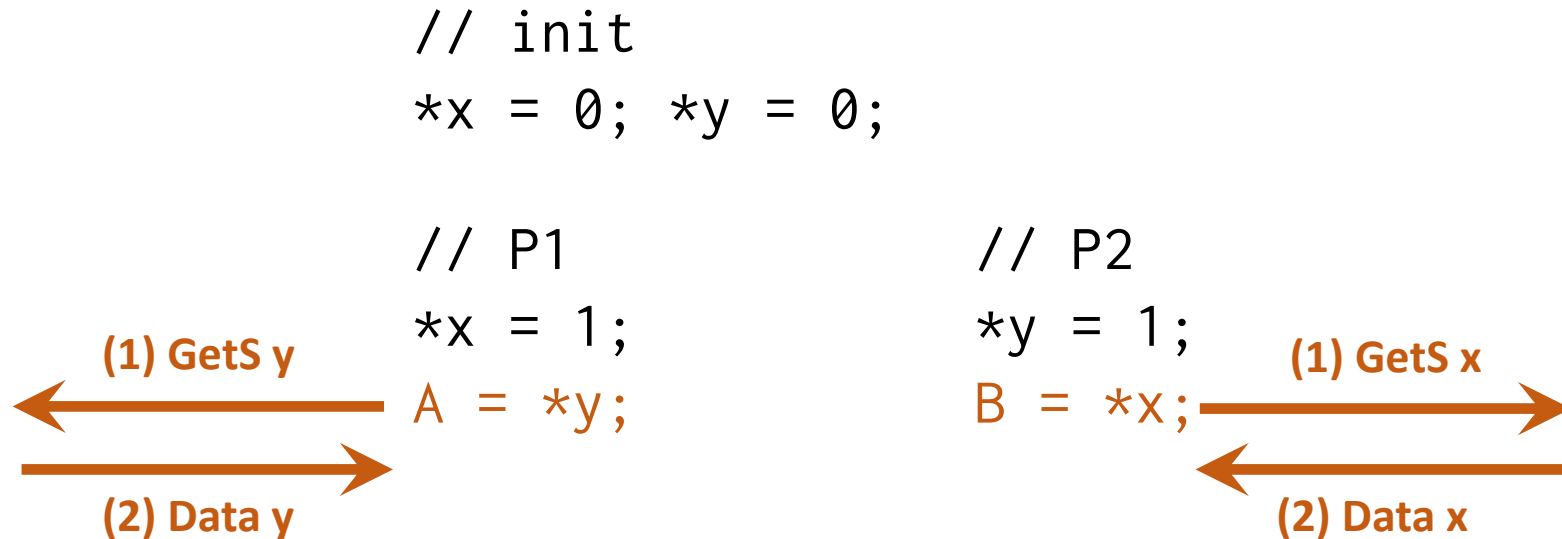
(1) GetS x →



Program Ordering – Example 5

Possible values: A, B?

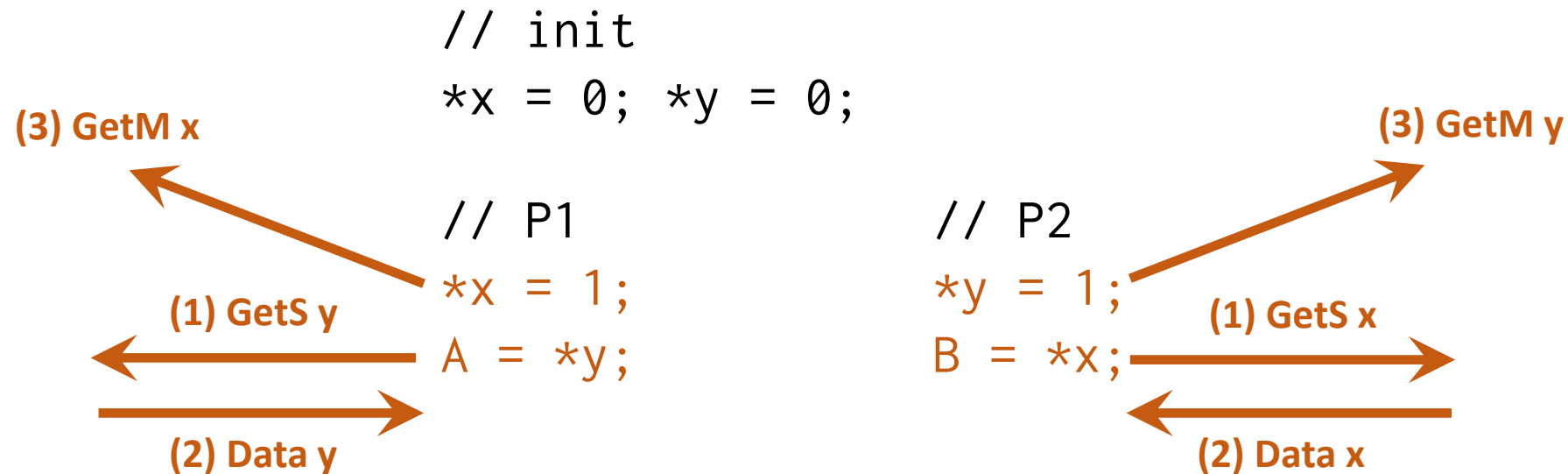
(A=0 B=1) (A=1 B=0) (A=1 B=1)



Program Ordering – Example 5

Possible values: A, B?

(A=0 B=1) (A=1 B=0) (A=1 B=1)

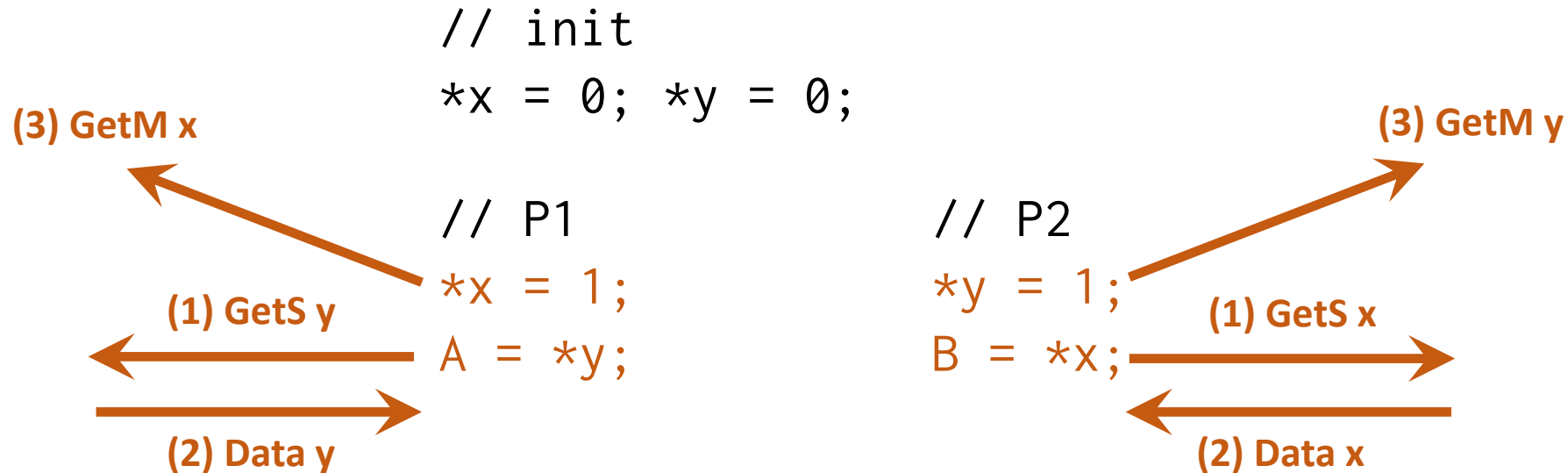


Program Ordering – Example 5

Possible values: A, B?

(A=0 B=1) (A=1 B=0) (A=1 B=1) **(A=0 B=0)**

SC violation!



Program Ordering – Example 6

Possible assertion fail?

```
// init
*x = 0; *y = 0;
```

```
// P1
*x = 1;
*y = 1;
```

```
// P2
if(*y != 0) {
    A = *x;
    assert(A != 0);
}
```

Program Ordering – Example 6

Possible assertion fail?

No

```
// init
*x = 0; *y = 0;
```

```
// P1
*x = 1;
*y = 1;
```

```
// P2
if(*y != 0) {
    A = *x;
    assert(A != 0);
}
```

Program Ordering – Example 6

Possible assertion fail?

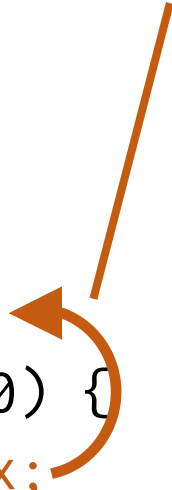
No

```
// init
*x = 0; *y = 0;
```

```
// P1
*x = 1;
*y = 1;
```

```
// P2
if(*y != 0) {
    A = *x;
    assert(A != 0);
}
```

possible if out-of-order execution
with branch prediction



Program Ordering – Example 6

Possible assertion fail?

Yes

SC violation!

```
// init  
*x = 0; *y = 0;
```

```
// P1  
*x = 1;  
*y = 1;
```

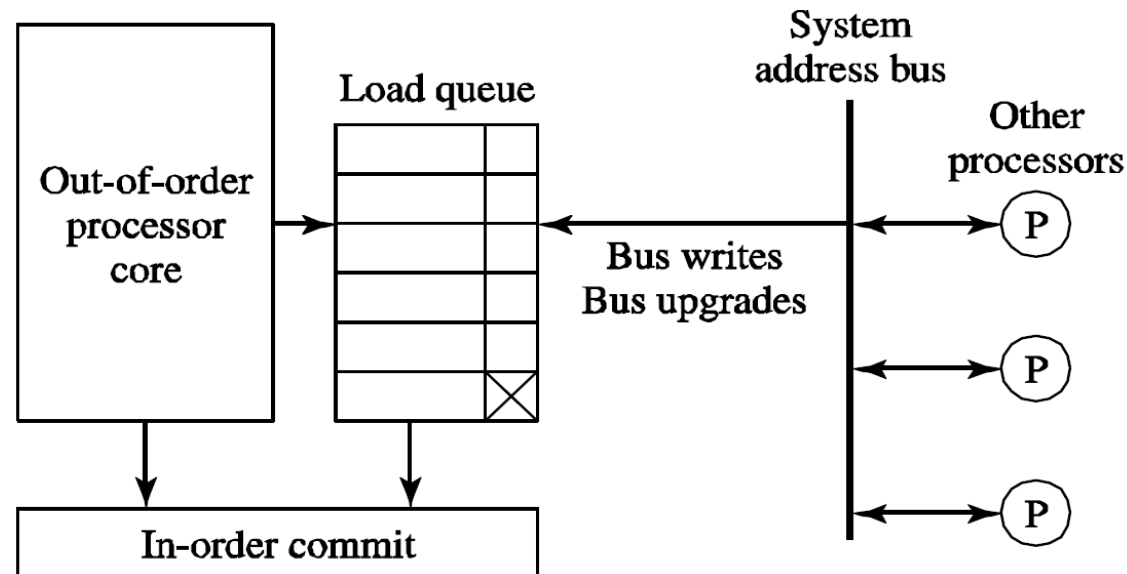
```
// P2  
if(*y != 0) {  
    A = *x;  
    assert(A != 0);  
}
```

possible if out-of-order execution
with branch prediction

Enforcing Program Ordering – Load Replay

Use load queue to record all in-flight (speculative) loads. Upon a coherence conflict (e.g., invalidation) from the memory hierarchy, mark conflicting loads for *replay*. At commit time, such loads are checked and replayed if necessary (which flushes dependent instructions).

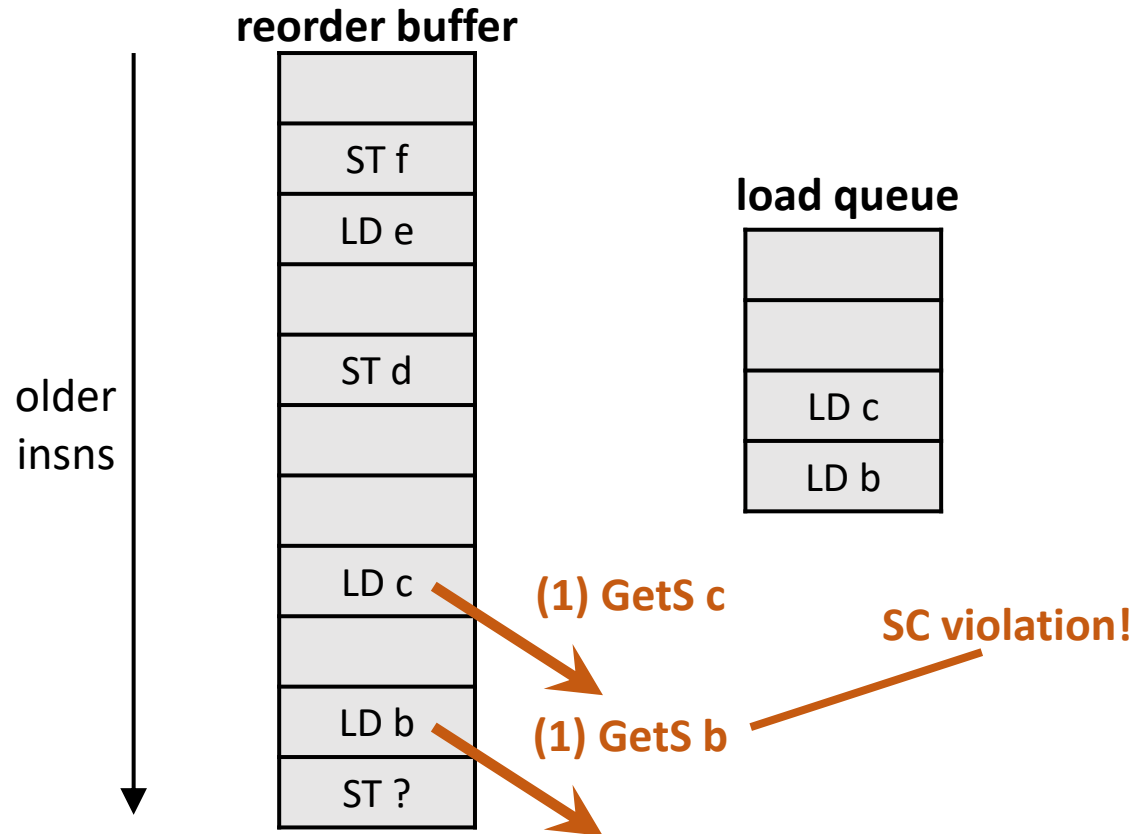
➤ **Takeaway:** Can reorder execution of loads/stores as long as invisible from other threads



Enforcing Program Ordering – Load Replay

Baseline processor with speculative loads:

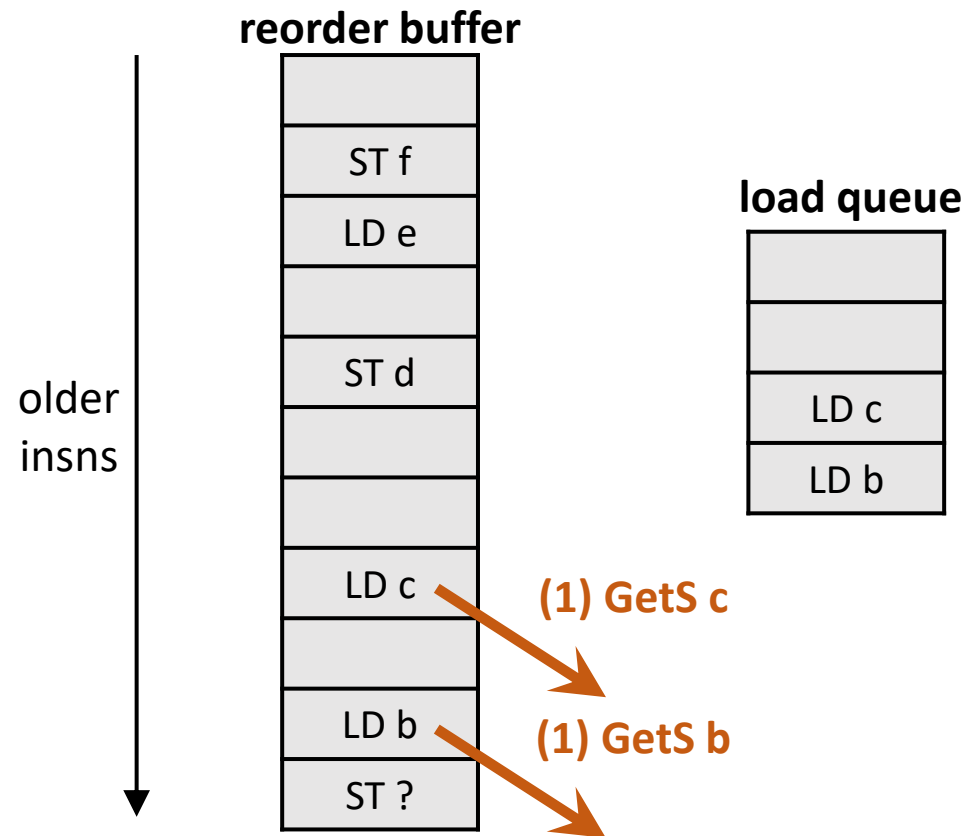
Loads and stores retire in order, but loads can execute before ambiguous stores



Enforcing Program Ordering – Load Replay

Load replay:

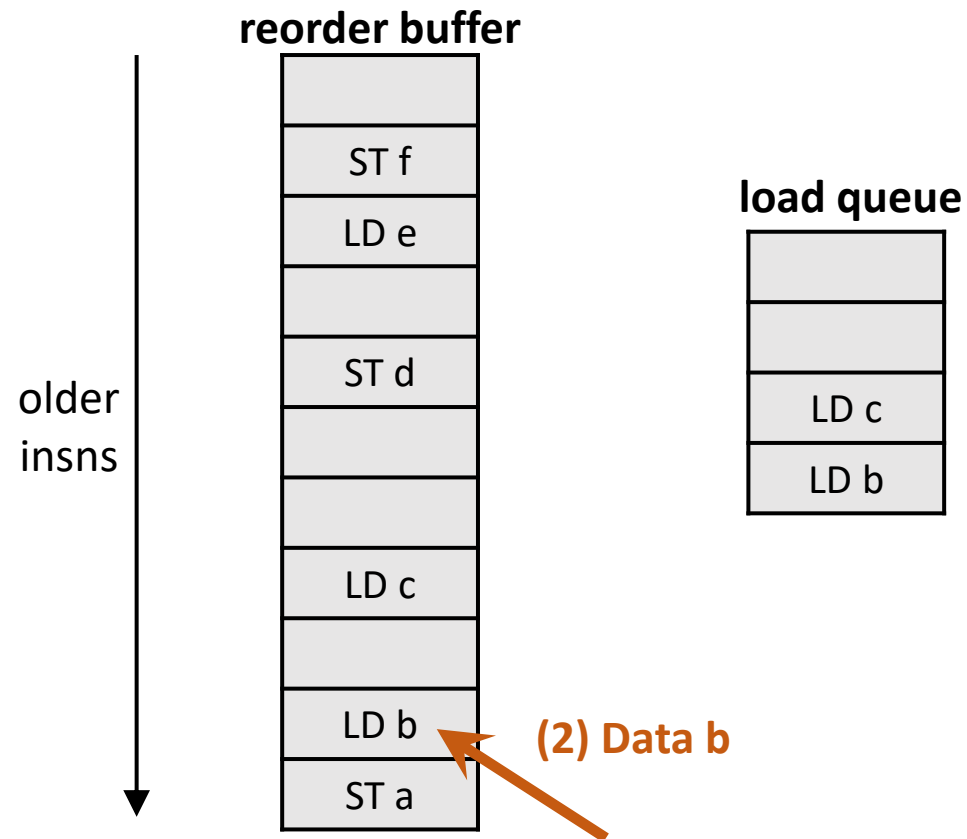
Allow independent loads to bypass but catch any invalidations and replay them if necessary



Enforcing Program Ordering – Load Replay

Load replay:

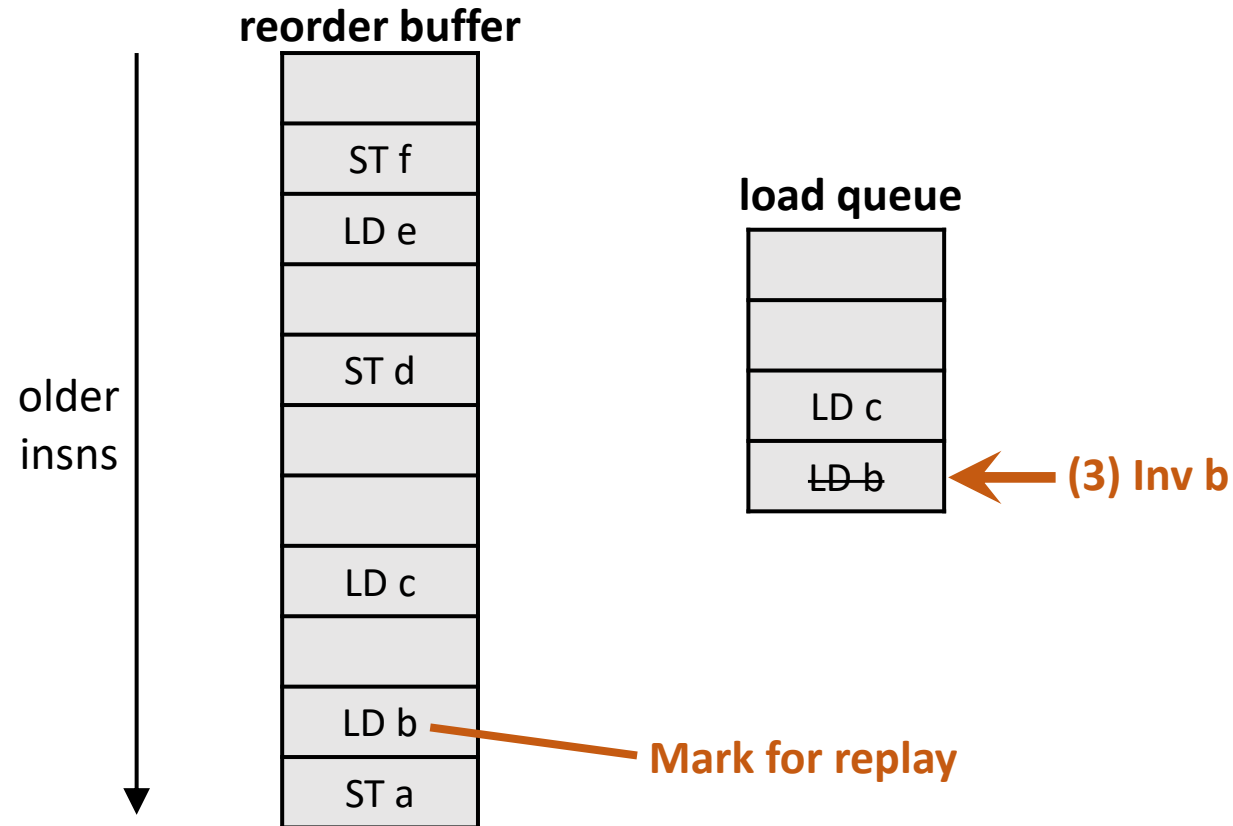
Allow independent loads to bypass but catch any invalidations and replay them if necessary



Enforcing Program Ordering – Load Replay

Load replay:

Allow independent loads to bypass but catch any invalidations and replay them if necessary



Enforcing Program Ordering – Example

Possible values: A, B?

```
// init
*x = 0; *y = 0;
```

```
// P1
*x = 1;
A = *y;
```

```
// P2
*y = 1;
B = *x;
```

Enforcing Program Ordering – Example

Possible values: A, B?

(A=0 B=1) (A=1 B=0) (A=1 B=1)

```
// init  
*x = 0; *y = 0;
```

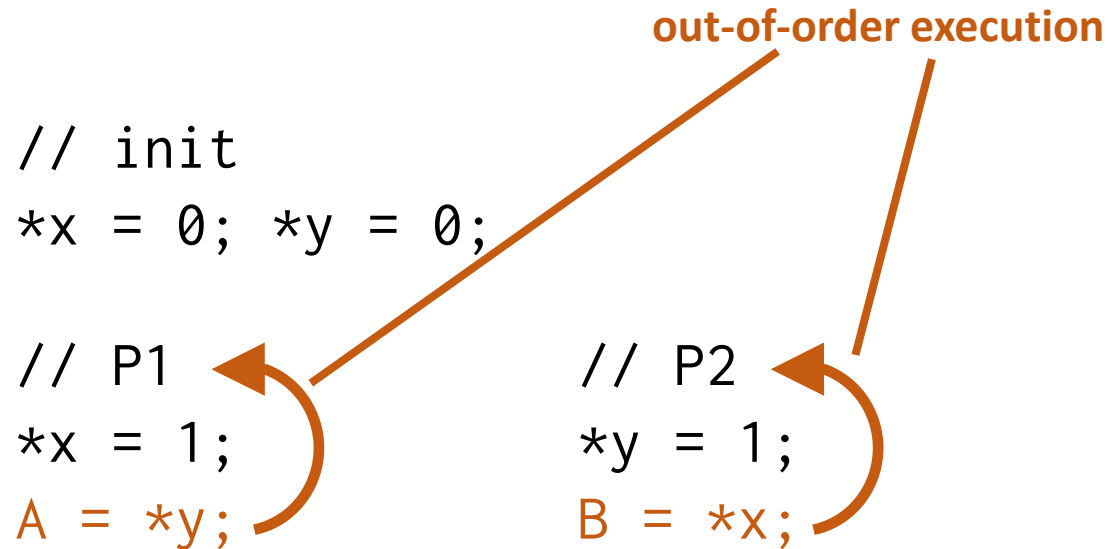
```
// P1  
*x = 1;  
A = *y;
```

```
// P2  
*y = 1;  
B = *x;
```

Enforcing Program Ordering – Example

Possible values: A, B?

(A=0 B=1) (A=1 B=0) (A=1 B=1)



Enforcing Program Ordering – Example

Possible values: A, B?

(A=0 B=1) (A=1 B=0) (A=1 B=1)

```
// init  
*x = 0; *y = 0;
```

```
    // P1  
    *x = 1;  
    A = *y;
```

← (1) GetS y

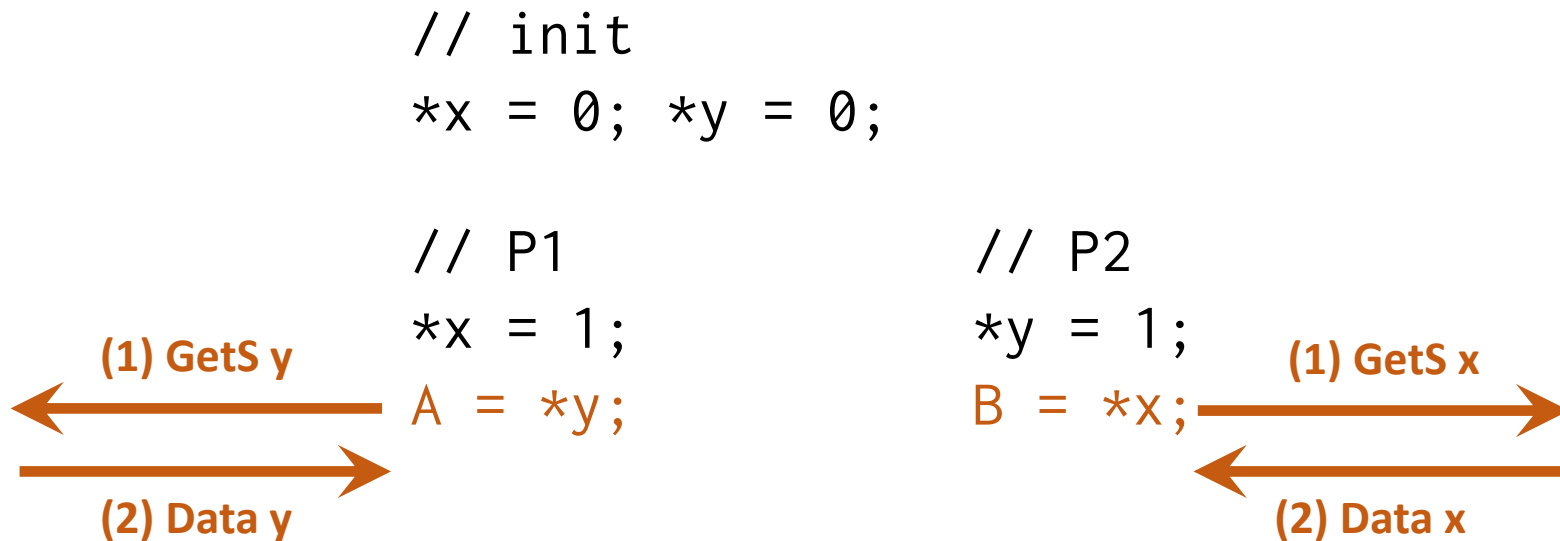
```
    // P2  
    *y = 1;  
    B = *x;
```

(1) GetS x →

Enforcing Program Ordering – Example

Possible values: A, B?

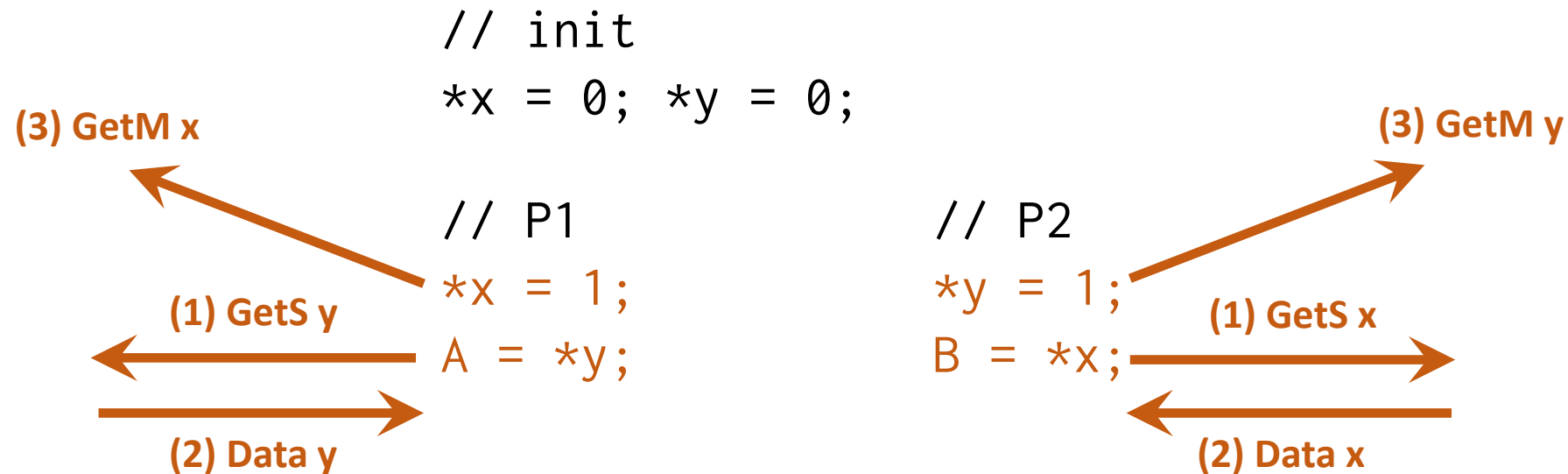
(A=0 B=1) (A=1 B=0) (A=1 B=1)



Enforcing Program Ordering – Example

Possible values: A, B?

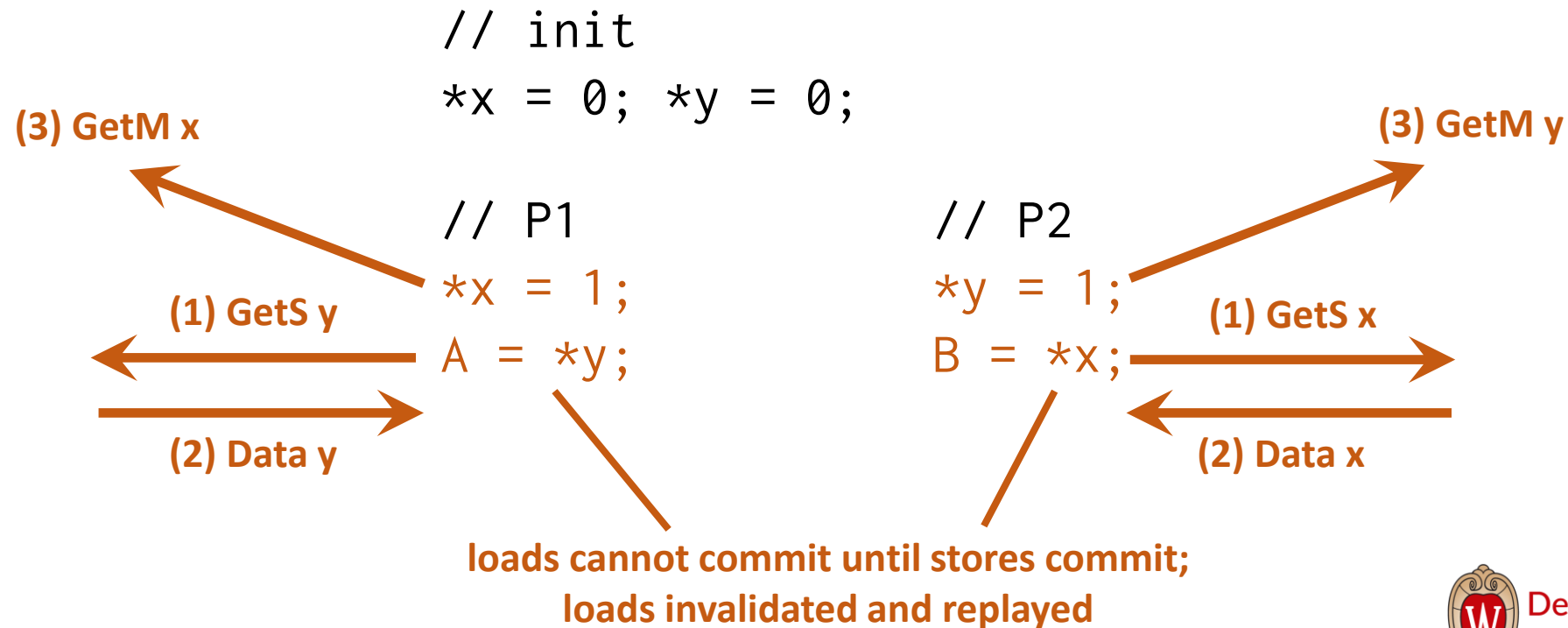
(A=0 B=1) (A=1 B=0) (A=1 B=1)



Enforcing Program Ordering – Example

Possible values: A, B?

(A=0 B=1) (A=1 B=0) (A=1 B=1)



Program Ordering – Example 7

Possible assertion fail?

No

```
// init
*x = 0; *y = 0;
```

```
// P1
*x = 1;
*y = 1;
```

```
// P2
if(*y != 0) {
    A = *x;
    assert(A != 0);
}
```

Program Ordering – Example 7

Possible assertion fail?

No

```
// init
*x = 0; *y = 0;
```

```
// P1
*x = 1;
*y = 1;
```

```
// P2
if(*y != 0) {
    A = *x;
    assert(A != 0);
}
```

NUCA/NUMA



directory y

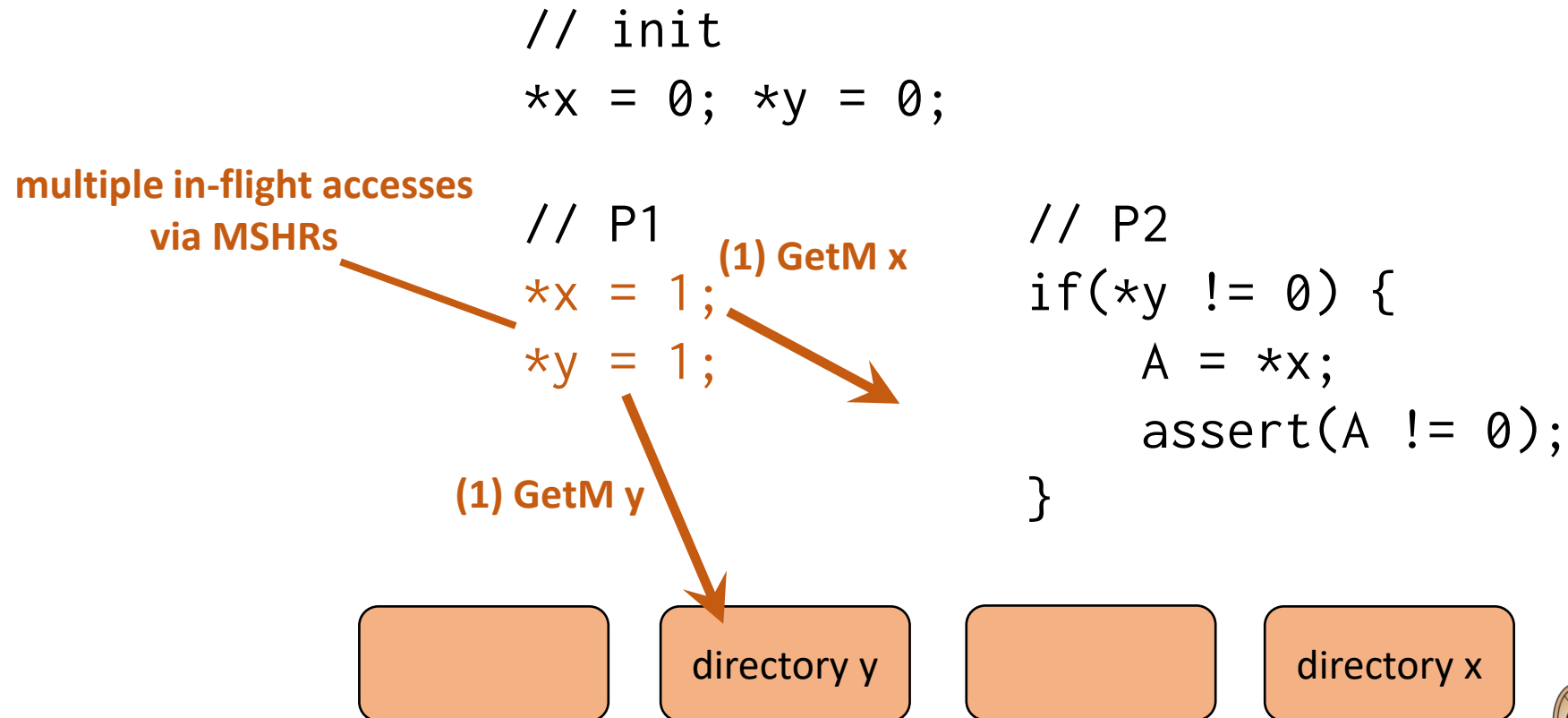
directory x



Program Ordering – Example 7

Possible assertion fail?

No



Program Ordering – Example 7

Possible assertion fail?

No

```
// init  
*x = 0; *y = 0;
```

```
// P1  
*x = 1;  
*y = 1;
```

(1) GetM x

(2) Data y

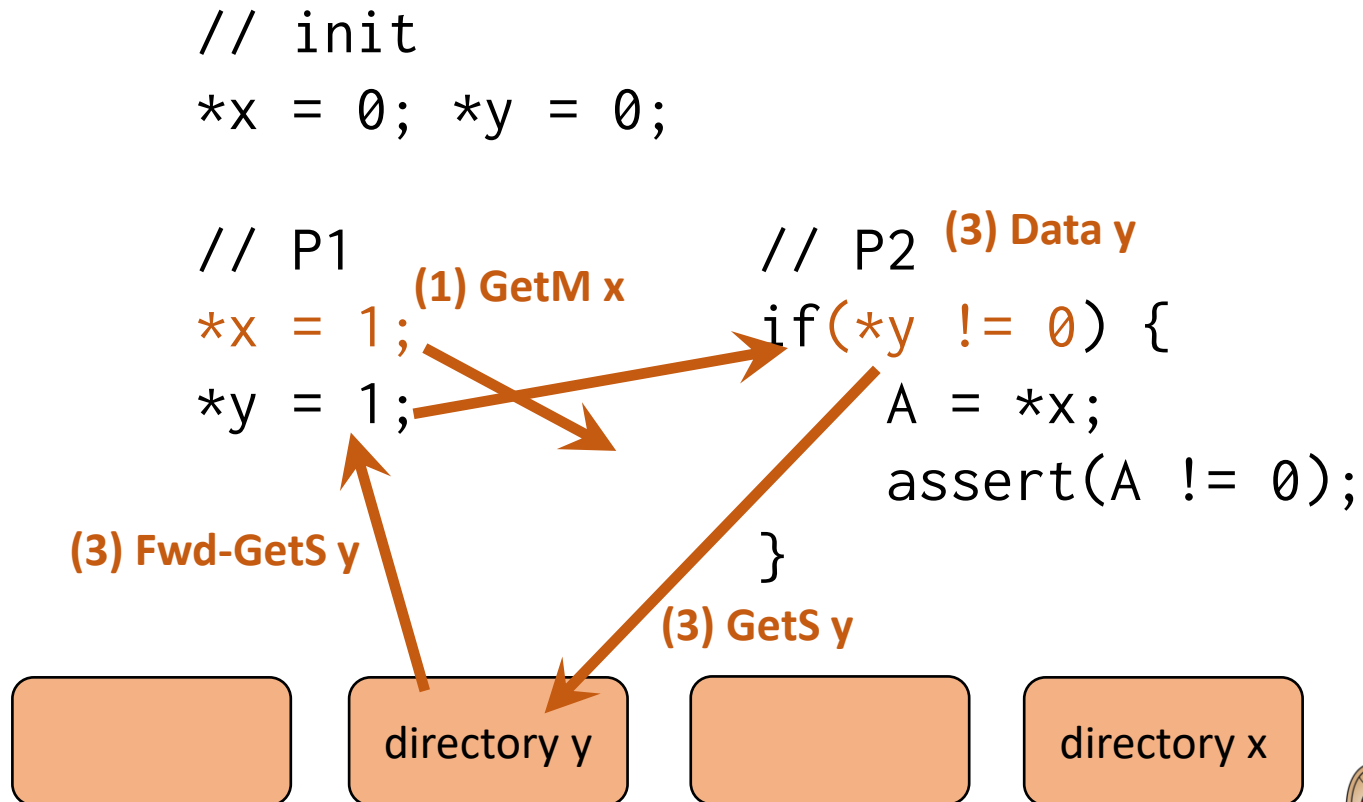
```
// P2  
if(*y != 0) {  
    A = *x;  
    assert(A != 0);  
}
```



Program Ordering – Example 7

Possible assertion fail?

No



Program Ordering – Example 7

Possible assertion fail?

No

```
// init  
*x = 0; *y = 0;
```

```
// P1  
*x = 1; (1) GetM x  
*y = 1;
```

```
// P2  
if(*y != 0) {  
    A = *x;  
    assert(A != 0);  
}  
(4) GetS x (4) Data x
```



Program Ordering – Example 7

Possible assertion fail?

Yes

SC violation!

```
// init
*x = 0; *y = 0;
```

```
// P1
*x = 1;
*y = 1;
```

(1) GetM x

```
// P2
if(*y != 0) {
  A = *x;
  assert(A != 0);
}
```

(4) GetS x

(4) Data x



directory y



directory x



Program Ordering – Example 8

Possible values: A?

```
// init
*z = 1; *x = z; *y = 0;

// P1                // P2
tmp = *x;            *y = 1;
A = *tmp;            *x = y;
```

Program Ordering – Example 8

Possible values: A?

(A=1)

```
// init
*z = 1; *x = z; *y = 0;
```

// P1	// P2
tmp = *x;	*y = 1;
A = *tmp;	*x = y;

Program Ordering – Example 8

Possible values: A?

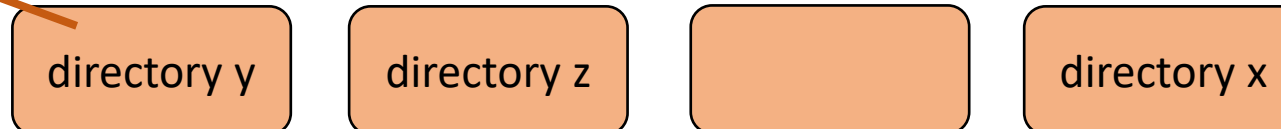
(A=1)

```
// init
*z = 1; *x = z; *y = 0;
```

```
// P1
tmp = *x;
A = *tmp;
```

```
// P2
*y = 1;
*x = y;
```

NUCA/NUMA



Program Ordering – Example 8

Possible values: A?

(A=1)

value prediction:
tmp = y

```
// init  
*z = 1; *x = z; *y = 0;
```

```
// P1  
tmp = *x;  
A = *tmp;
```

```
// P2  
*y = 1;  
*x = y;
```

directory y

directory z

directory x



Program Ordering – Example 8

Possible values: A?

(A=1)

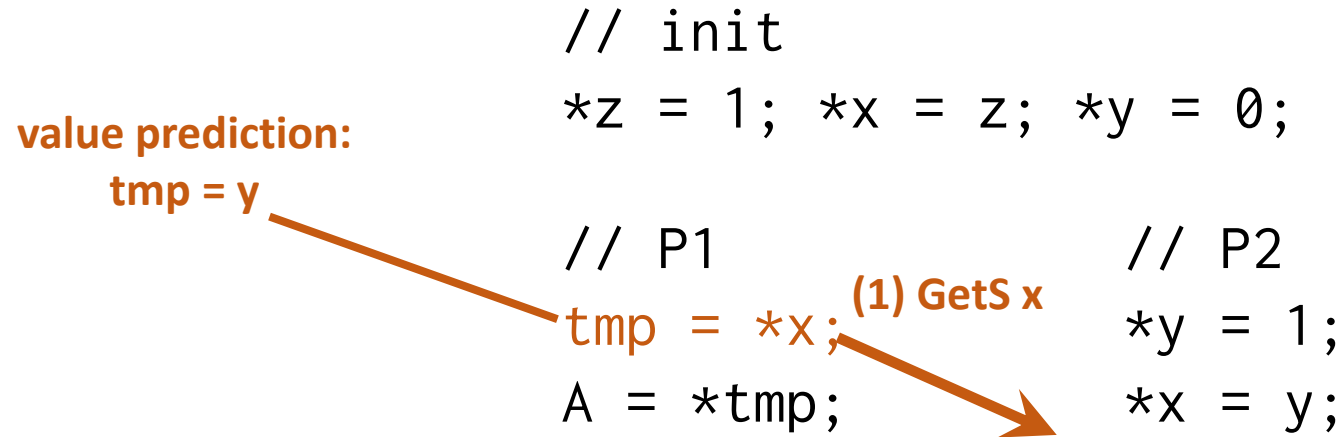
value prediction:
tmp = y

```
// init
*z = 1; *x = z; *y = 0;

// P1
tmp = *x;
A = *tmp;

// P2
*y = 1;
*x = y;
```

(1) GetS x



directory y

directory z

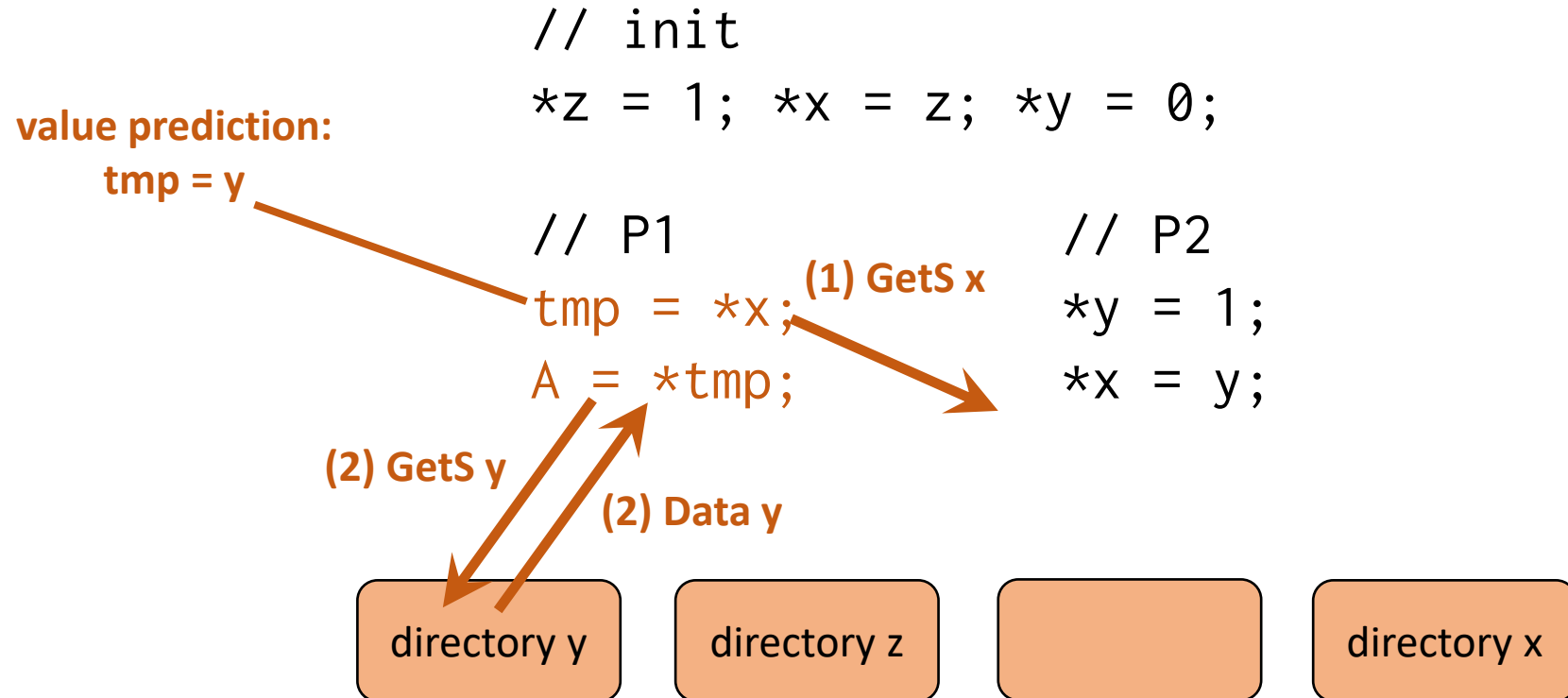
directory x



Program Ordering – Example 8

Possible values: A?

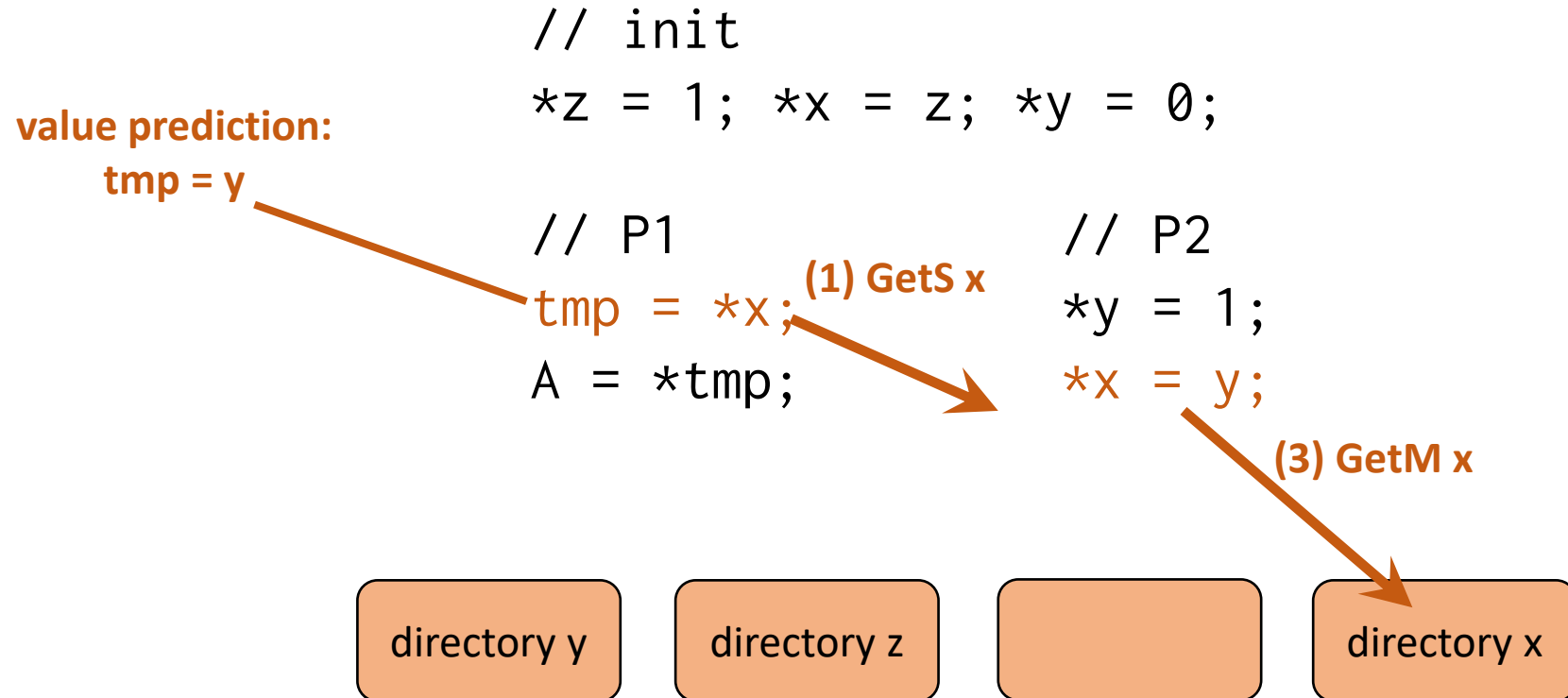
(A=1)



Program Ordering – Example 8

Possible values: A?

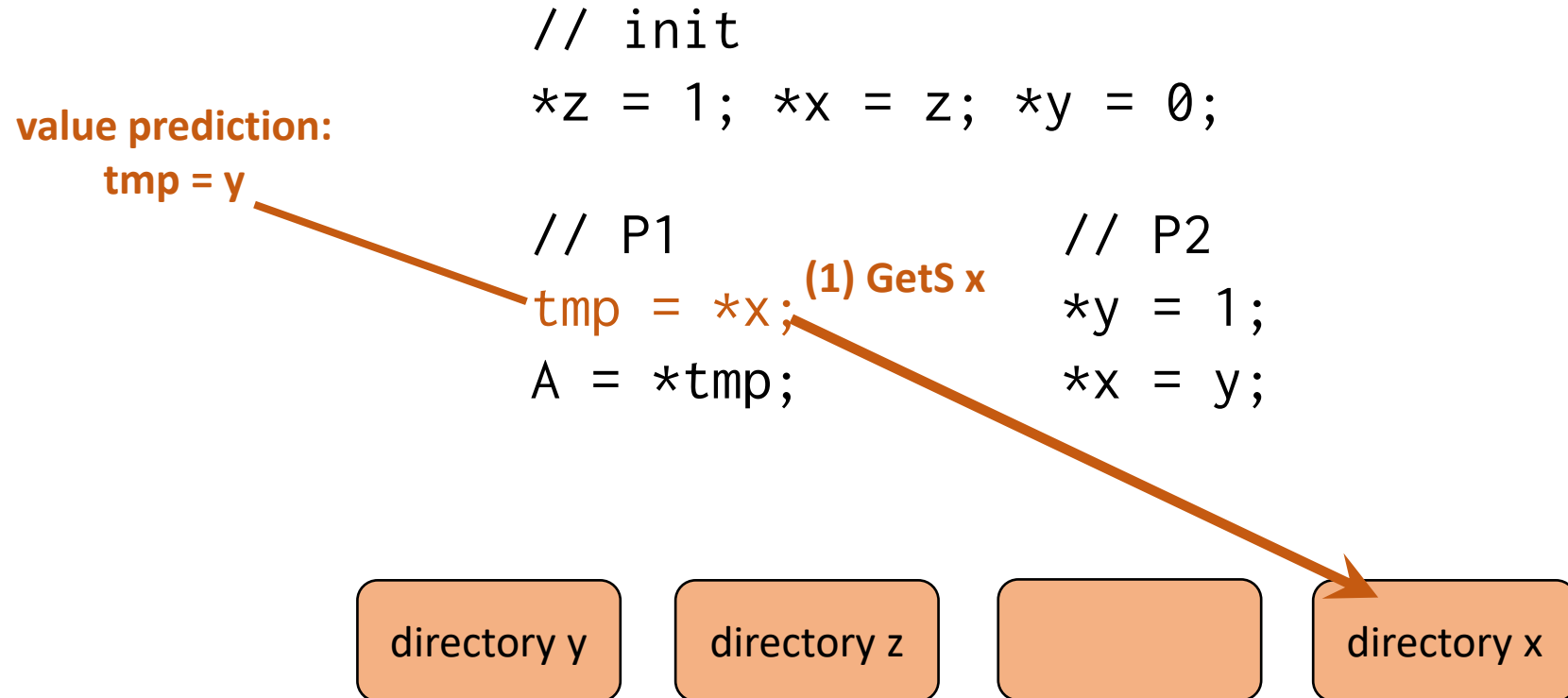
(A=1)



Program Ordering – Example 8

Possible values: A?

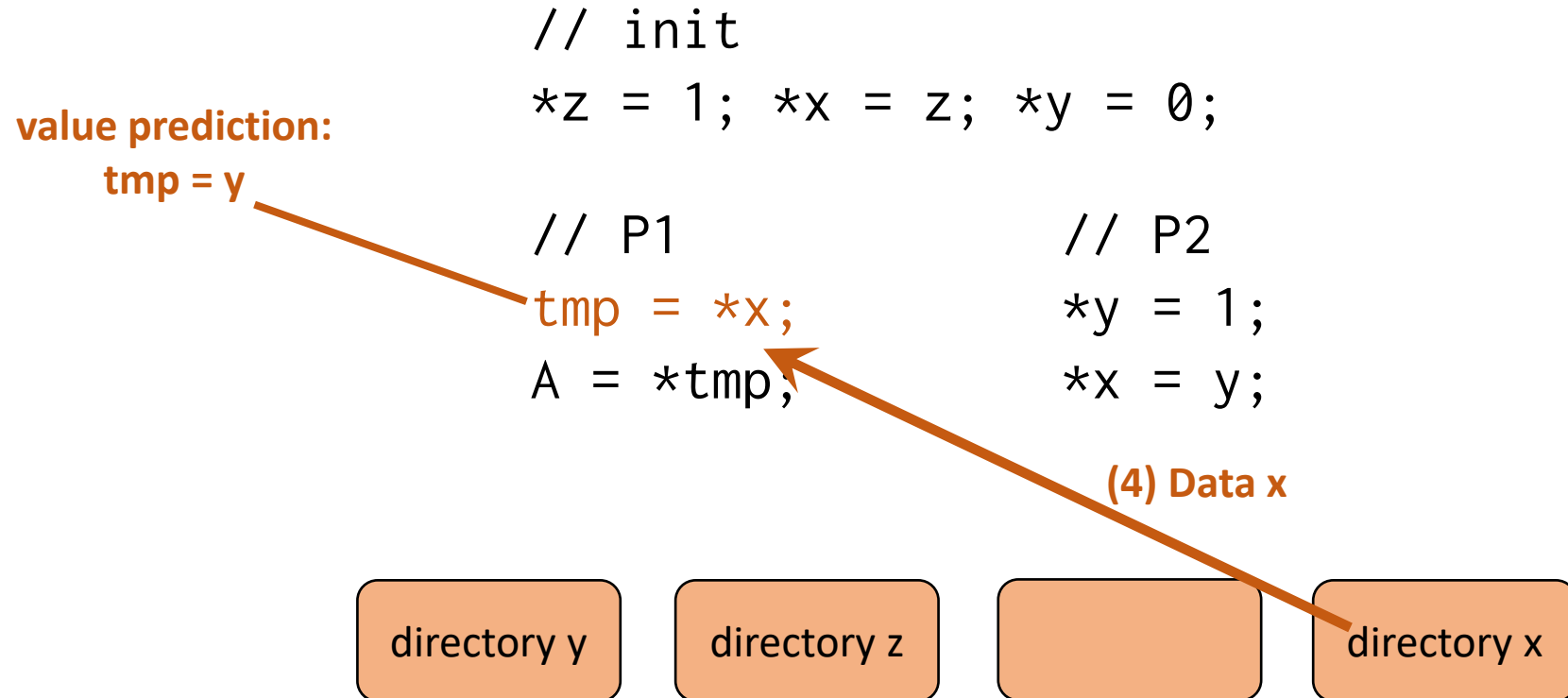
(A=1)



Program Ordering – Example 8

Possible values: A?

(A=1)



Program Ordering – Example 8

Possible values: A?

(A=1)

value prediction:
tmp = y
prediction correct!

```
// init
*z = 1; *x = z; *y = 0;

// P1
tmp = *x;
A = *tmp;

// P2
*y = 1;
*x = y;
```

directory y

directory z

directory x



Program Ordering – Example 8

Possible values: A?

(A=1) (A=0)

SC violation!

```
// init
*z = 1; *x = z; *y = 0;
```

value prediction:

tmp = y

prediction correct!

```
// P1
tmp = *x;
A = *tmp;
```

```
// P2
*y = 1;
*x = y;
```

directory y

directory z

directory x



Enforcing Program Ordering – Memory Fences

Memory fences (or barriers) are special instructions that allow compilers/programmers to manually enforce ordering of memory reads and writes within a thread. Atomic instructions implicitly invoke fences.

mfence: All prior loads and stores are performed before proceeding

lfence: All prior loads are performed before proceeding

sfence: All prior stores are performed before proceeding

Enforcing Program Ordering – Example

Possible assertion fail?

```
// init
*x = 0; *y = 0;
```

```
// P1
*x = 1;
*y = 1;
```

```
// P2
if(*y != 0) {
    A = *x;
    assert(A != 0);
}
```

Enforcing Program Ordering – Example

Possible assertion fail?

```
// init
*x = 0; *y = 0;
```

```
// P1
*x = 1;
sfence;
*y = 1;
```

```
// P2
if(*y != 0) {
    lfence;
    A = *x;
    assert(A != 0);
}
```

Enforcing Program Ordering – Example

Possible assertion fail?

```
// init
*x = 0; *y = 0;
```

```
// P1
*x = 1; (1) GetM x
sfence;
*y = 1;
```

```
// P2
if(*y != 0) {
    lfence;
    A = *x;
    assert(A != 0);
}
```



directory y



directory x



Enforcing Program Ordering – Example

Possible assertion fail?

```
// init
*x = 0; *y = 0;
```

```
// P1
*x = 1; (1) GetM x
sfence;
*y = 1;
```

can't issue GetM y until
all prior stores committed

```
// P2
if(*y != 0) {
    lfence;
    A = *x;
    assert(A != 0);
}
```



Enforcing Program Ordering – Example

Possible assertion fail?

No

```
// init
*x = 0; *y = 0;
```

```
// P1
*x = 1; (1) GetM x
sfence;
*y = 1;
```

can't issue GetM y until
all prior stores committed

```
// P2
if(*y != 0) {
    lfence;
    A = *x;
    assert(A != 0);
}
```



Enforcing Program Ordering

How many memory fences needed to ensure SC?

```
// init  
*x = 0; *y = 0;
```

```
// P1  
*x = 1;  
A = *y;
```

```
// P2  
*y = 1;  
B = *x;
```

Enforcing Program Ordering

How many memory fences needed to ensure SC?

2

```
// init  
*x = 0; *y = 0;
```

```
// P1  
*x = 1;  
mfence;  
A = *y;
```

```
// P2  
*y = 1;  
mfence;  
B = *x;
```

Sequential Consistency – Store Atomicity

Requirements for a sequentially consistent processor:

Program Ordering:

Each individual core's reads and writes must be globally performed in program order.

Store Atomicity:

If a write is performed with respect to core P_i before another write is performed with respect to P_i , the two writes must be performed in the same order with respect to any other core P_j .

- Property that states the existence of a total order of all changes to memory state (i.e., writes)

Store Atomicity

Possible: A=1, B=1, C=0, D=0?

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
*x = 1;
```

```
// P2
```

```
*y = 1;
```

```
// P3
```

```
A = *x;
```

```
C = *y;
```

```
// P4
```

```
B = *y;
```

```
D = *x;
```



Store Atomicity

Possible: A=1, B=1, C=0, D=0?

```
// init  
*x = 0; *y = 0;
```

```
// P1  
*x = 1;
```

```
// P2  
*y = 1;
```

with store atomicity:
must agree on order of
writes to x and y

```
// P3  
A = *x;  
C = *y;
```

```
// P4  
B = *y;  
D = *x;
```



Store Atomicity

Possible: A=1, B=1, C=0, D=0?

```
// init
*x = 0; *y = 0;
```

```
// P1
*x = 1;
```

```
// P2
*y = 1;
```

```
// P3
A = *x;
C = *y;
```

```
// P4
B = *y;
D = *x;
```

with store atomicity:

```
*x = 1;
    ↓
*y = 1;
```

Store Atomicity

Possible: A=1, B=1, C=0, D=0? No

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
*x = 1;
```

```
// P2
```

```
*y = 1;
```

```
// P3
```

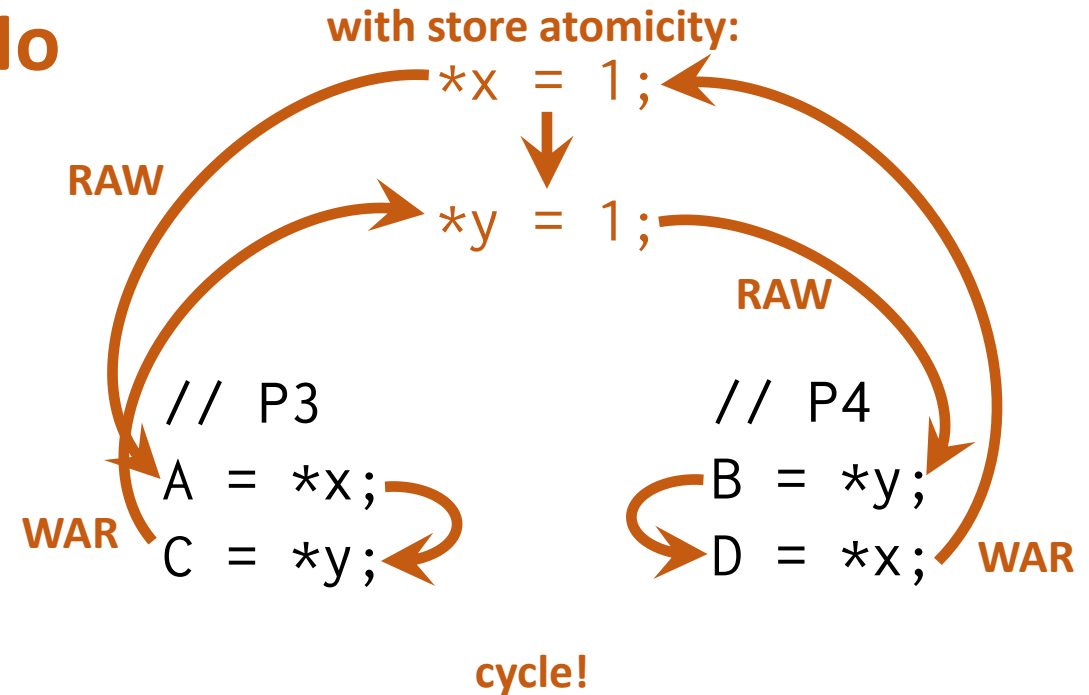
```
A = *x;
```

```
C = *y;
```

```
// P4
```

```
B = *y;
```

```
D = *x;
```



Store Atomicity

Possible: A=1, B=1, C=0, D=0?

```
// init  
*x = 0; *y = 0;
```

```
// P1  
*x = 1;
```

```
// P2  
*y = 1;
```

```
// P3  
A = *x;  
C = *y;
```

**y = 1;*
↓
**x = 1;*

```
// P4  
B = *y;  
D = *x;
```

with store atomicity:

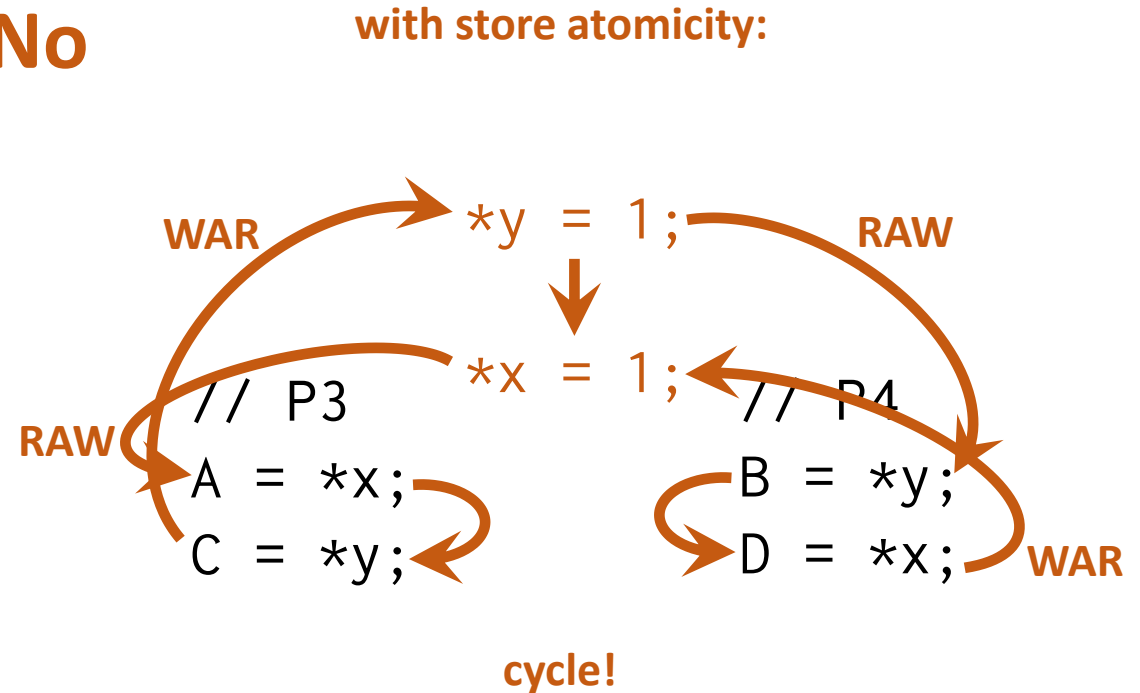
Store Atomicity

Possible: A=1, B=1, C=0, D=0? **No**

```
// init  
*x = 0; *y = 0;
```

```
// P1  
*x = 1;
```

```
// P2  
*y = 1;
```



Store Atomicity

Possible: A=1, B=1, C=0, D=0?

```
// init  
*x = 0; *y = 0;
```

```
// P1  
*x = 1;
```

```
// P2  
*y = 1;
```

without store atomicity:
can disagree on order of
writes to x and y

```
// P3  
A = *x;  
C = *y;
```

```
// P4  
B = *y;  
D = *x;
```

Store Atomicity

Possible: A=1, B=1, C=0, D=0?

```
// init
*x = 0; *y = 0;
```

```
// P1
*x = 1;
```

```
// P2
*y = 1;
```

```
// P3
A = *x;
C = *y;
```

```
// P4
B = *y;
D = *x;
```

without store atomicity:

```
*x = 1;
↓
*y = 1;
↓
*x = 1;
```

Store Atomicity

Possible: A=1, B=1, C=0, D=0?

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
*x = 1;
```

```
// P2
```

```
*y = 1;
```

```
// P3
```

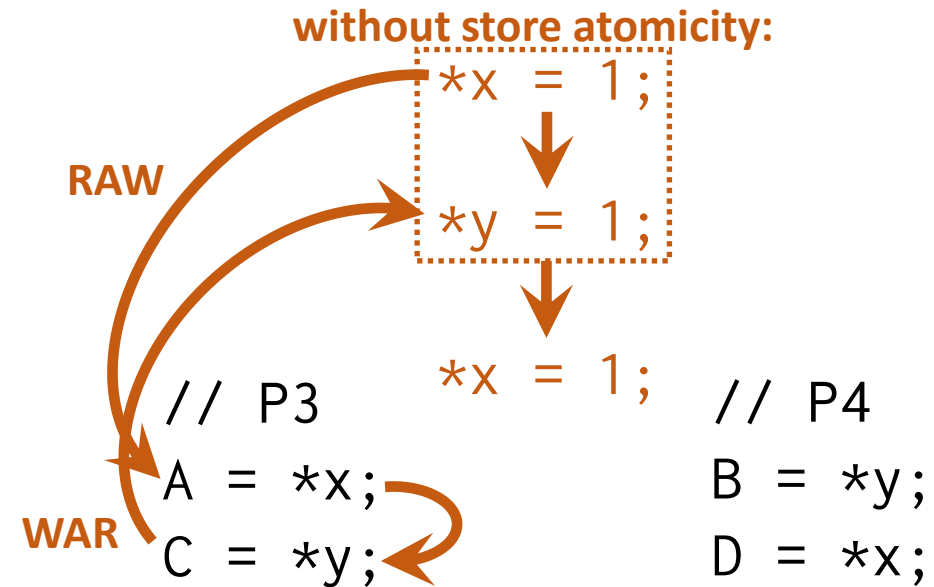
```
A = *x;
```

```
C = *y;
```

```
// P4
```

```
B = *y;
```

```
D = *x;
```



Store Atomicity

Possible: A=1, B=1, C=0, D=0?

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
*x = 1;
```

```
// P2
```

```
*y = 1;
```

```
// P3
```

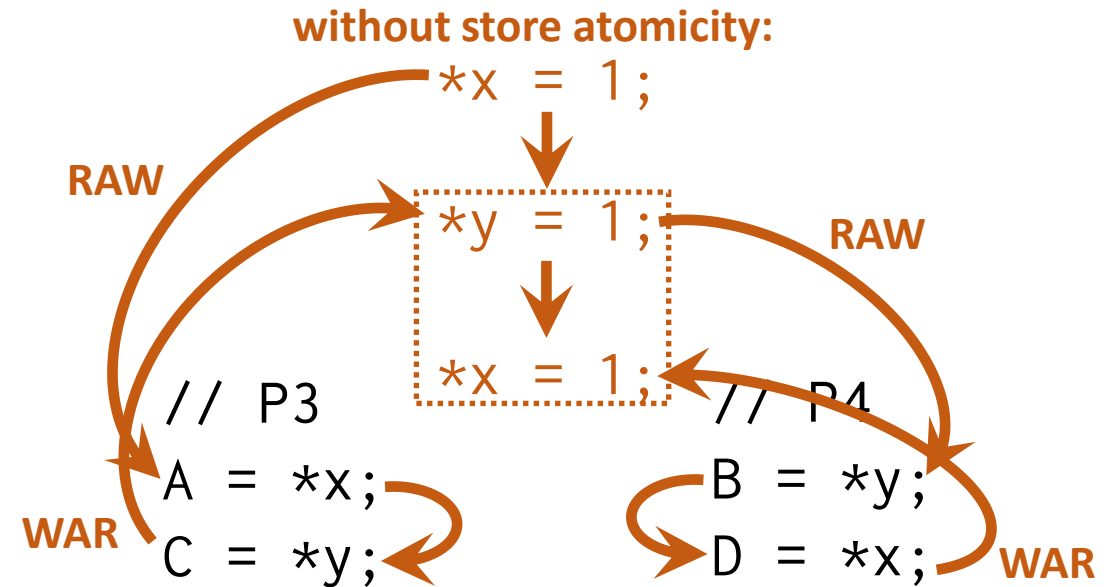
```
A = *x;
```

```
C = *y;
```

```
// P4
```

```
B = *y;
```

```
D = *x;
```



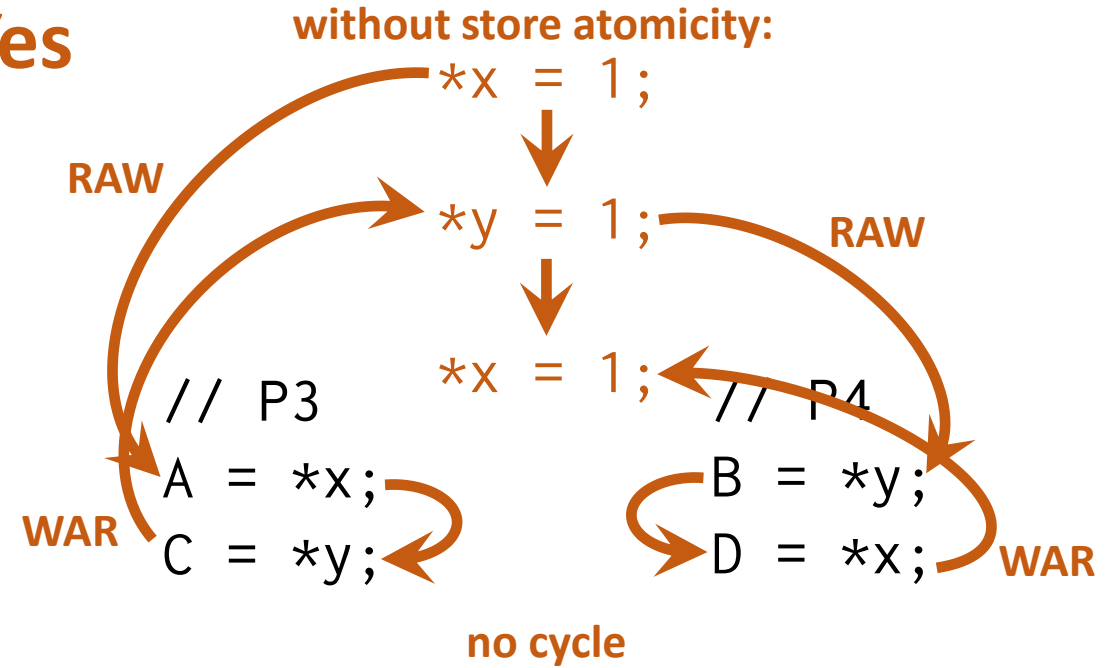
Store Atomicity

Possible: A=1, B=1, C=0, D=0? Yes

```
// init  
*x = 0; *y = 0;
```

```
// P1  
*x = 1;
```

```
// P2  
*y = 1;
```



Store Atomicity

How many memory fences needed to ensure SC?

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
*x = 1;
```

```
// P2
```

```
*y = 1;
```

```
// P3
```

```
A = *x;
```

```
C = *y;
```

```
// P4
```

```
B = *y;
```

```
D = *x;
```

Store Atomicity

How many memory fences needed to ensure SC?
2?

```
// init  
*x = 0; *y = 0;
```

```
// P1  
*x = 1;
```

```
// P2  
*y = 1;
```

```
// P3  
A = *x;  
mfence;  
C = *y;
```

```
// P4  
B = *y;  
mfence;  
D = *x;
```

Store Atomicity

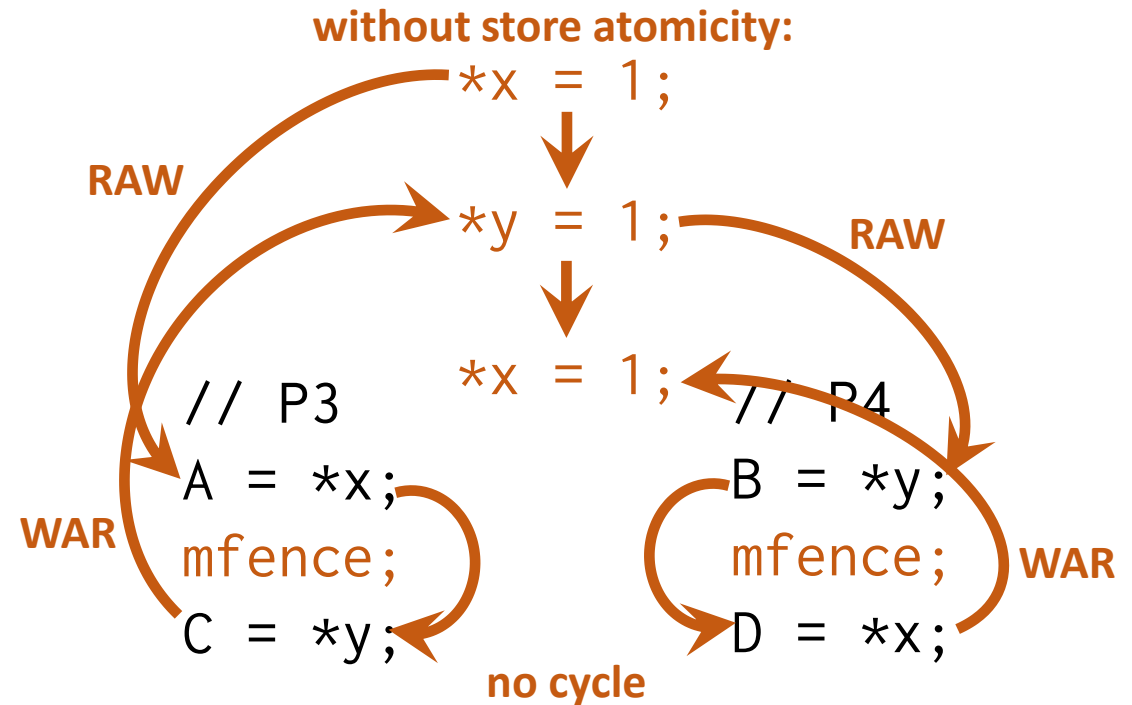
How many memory fences needed to ensure SC?

2?

```
// init
*x = 0; *y = 0;
```

```
// P1
*x = 1;
```

```
// P2
*y = 1;
```



Store Atomicity

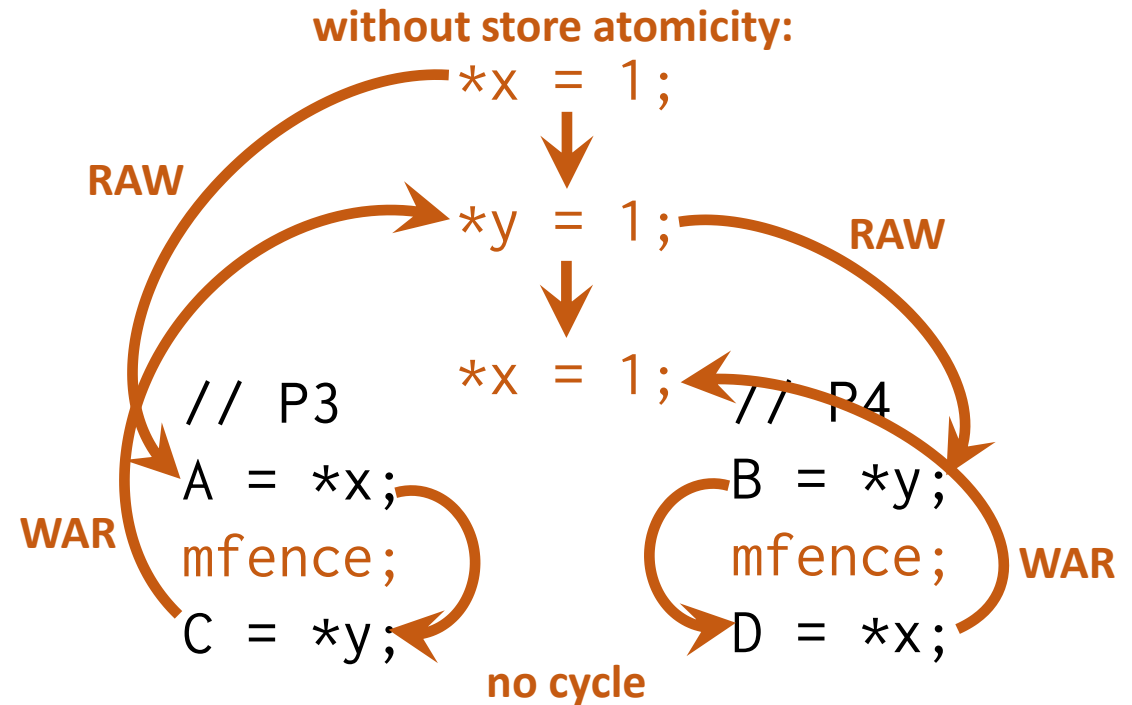
How many memory fences needed to ensure SC?

Fences alone not enough

```
// init
*x = 0; *y = 0;
```

```
// P1
*x = 1;
```

```
// P2
*y = 1;
```



Enforcing Store Atomicity

Requirements for a sequentially consistent processor:

Program Ordering:

Each individual core's reads and writes must be globally performed in program order.

Store Atomicity:

If a write is performed with respect to core P_i before another write is performed with respect to P_i , the two writes must be performed in the same order with respect to any other core P_j .

- Property that states the existence of a total order of all changes to memory state (i.e., writes)
- Therefore, need *write propagation and write serialization*

Enforcing Store Atomicity

Snooping-Based Coherence:

All stores are atomically globally performed via bus/broadcasts

Update-Based Coherence:

Update messages must be ordered across all nodes

Invalidation-Based Coherence:

Writers must not reveal new values until all invalidations acknowledged

Enforcing Store Atomicity – Example

Possible values: A?

```
// init  
*x = 0; *y = 0;
```

```
// P1  
*x = 1;
```

```
// P2  
while(*x == 0);  
*y = 1;
```

```
// P3  
while(*y == 0);  
A = *x;
```

Enforcing Store Atomicity – Example

Possible values: A?

(A=1)

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
*x = 1;
```

```
// P2
```

```
while(*x == 0);
```

```
*y = 1;
```

```
// P3
```

```
while(*y == 0);
```

```
A = *x;
```



Enforcing Store Atomicity – Example

Possible values: A?

(A=1)

```
// init
*x = 0; *y = 0;
```

```
// P1
*x = 1;
```

(1) GetM x



directory x

```
// P2
while(*x == 0);
*y = 1;
```

directory y

```
// P3
while(*y == 0);
A = *x;
```



Enforcing Store Atomicity – Example

Possible values: A?

(A=1)

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
*x = 1;
```

```
// P2
```

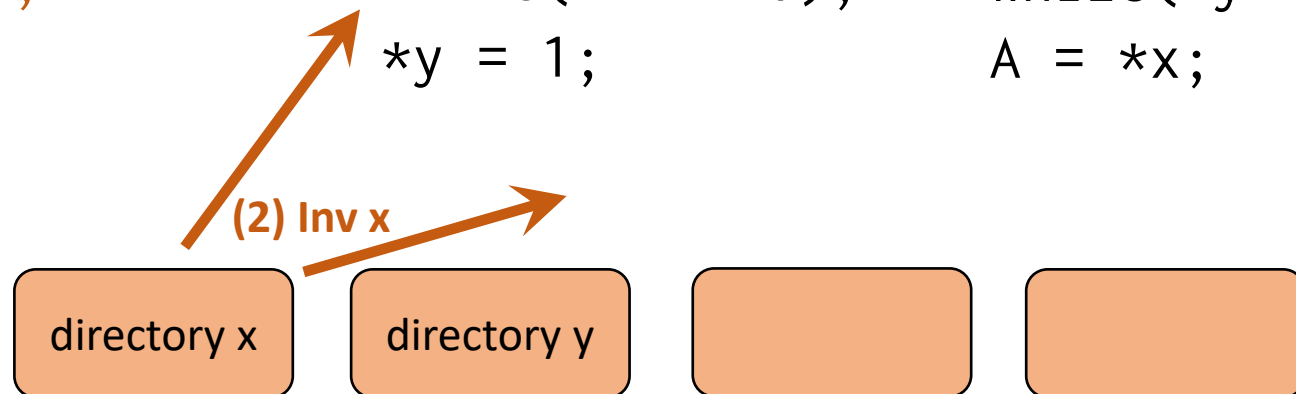
```
while(*x == 0);
```

```
*y = 1;
```

```
// P3
```

```
while(*y == 0);
```

```
A = *x;
```



Enforcing Store Atomicity – Example

Possible values: A?

(A=1)

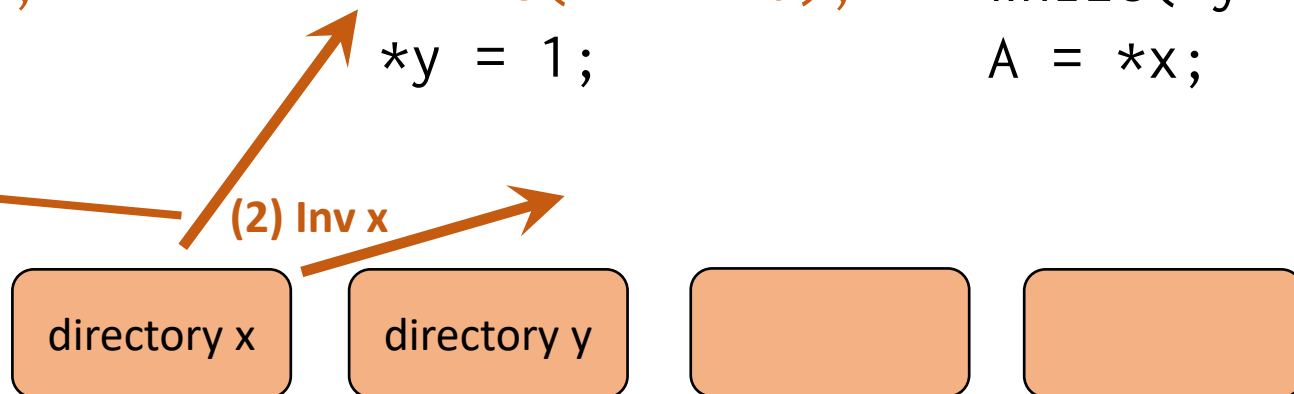
```
// init  
*x = 0; *y = 0;
```

```
// P1  
*x = 1;
```

```
// P2  
while(*x == 0);  
*y = 1;
```

```
// P3  
while(*y == 0);  
A = *x;
```

what if Inv x gets to P2
much earlier than P3?



Enforcing Store Atomicity – Example

Possible values: A?

(A=1)

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
*x = 1;
```

```
// P2
```

```
while(*x == 0);
```

```
*y = 1;
```

(3) GetM y

```
// P3
```

```
while(*y == 0);
```

```
A = *x;
```

(2) Inv x

(4) Inv y



Enforcing Store Atomicity – Example

Possible values: A?

(A=1) (A=0)

SC violation!

```
// init  
*x = 0; *y = 0;
```

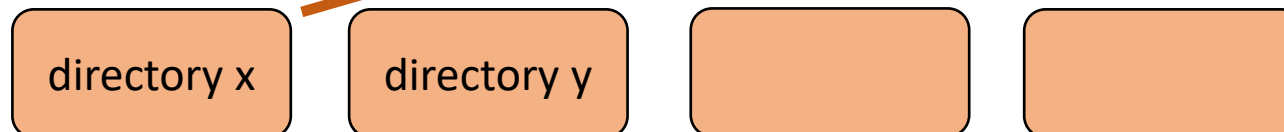
```
// P1  
*x = 1;
```

```
// P2  
while(*x == 0);  
*y = 1;
```

```
// P3  
while(*y == 0);  
A = *x;
```

private cache hit

(2) Inv x



Enforcing Store Atomicity – Example

Possible values: A?

(A=1)

```
// init  
*x = 0; *y = 0;
```

```
// P1  
*x = 1;
```

```
// P2  
while(*x == 0);  
*y = 1;
```

```
// P3  
while(*y == 0);  
A = *x;
```

coherence protocol must
ensure store atomicity

what if Inv x gets to P2
much earlier than P3?

(2) Inv x

directory x

directory y



Enforcing Store Atomicity – Example

Possible values: A?

(A=1)

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
*x = 1;
```

```
// P2
```

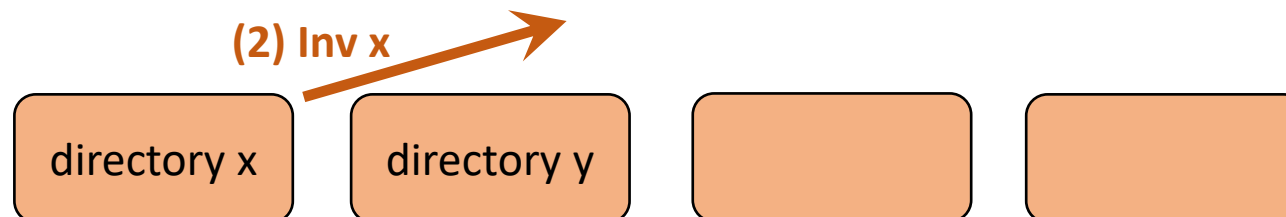
```
while(*x == 0);
```

```
*y = 1;
```

```
// P3
```

```
while(*y == 0);
```

```
A = *x;
```



Enforcing Store Atomicity – Example

Possible values: A?

(A=1)

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
*x = 1;
```

```
// P2
```

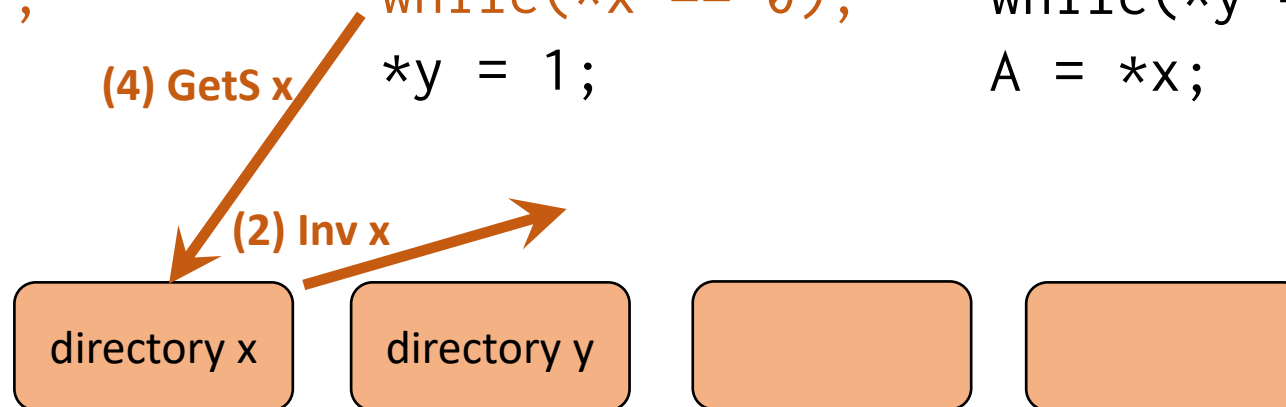
```
while(*x == 0);
```

```
*y = 1;
```

```
// P3
```

```
while(*y == 0);
```

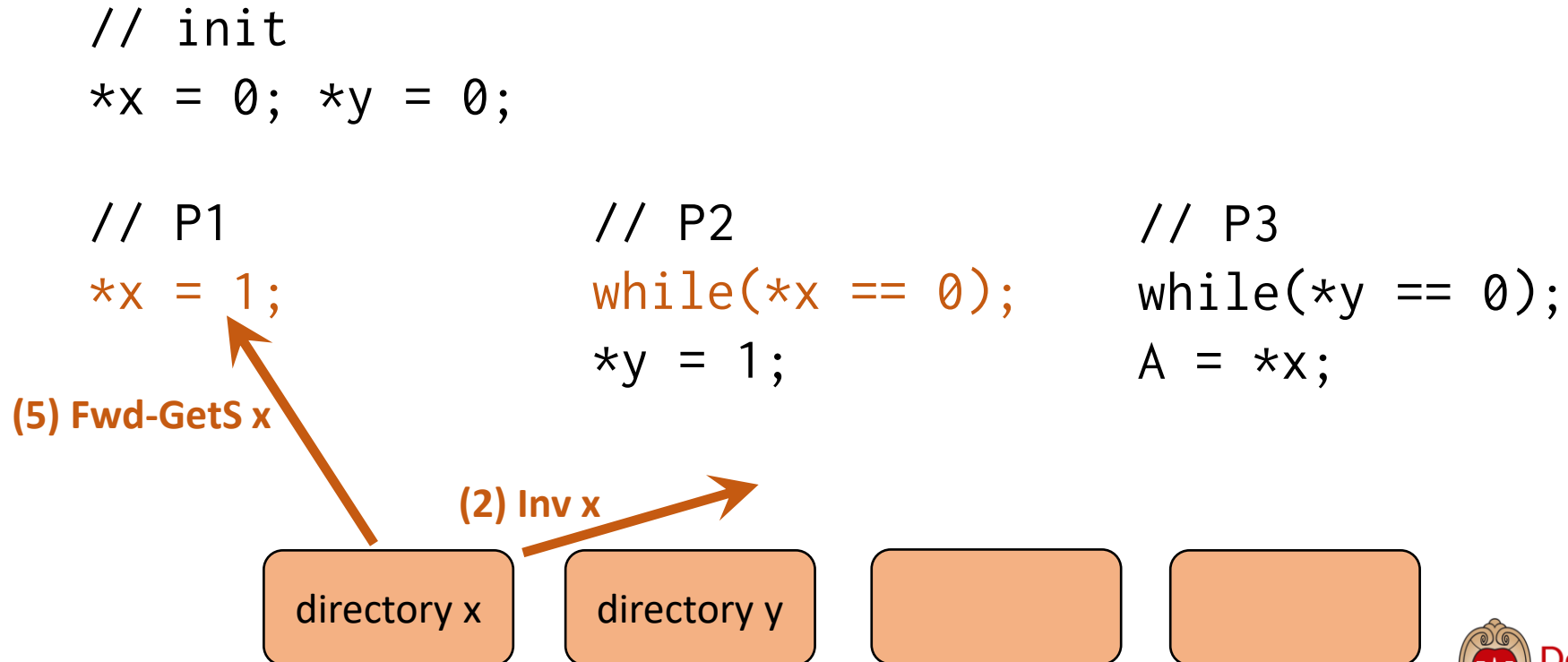
```
A = *x;
```



Enforcing Store Atomicity – Example

Possible values: A?

(A=1)



Enforcing Store Atomicity – Example

Cache Controller

Current state	Load	Store	Evict	Fwd-GetS	Fwd-GetM	Inv	Put-Ack	Data from dir [ack=0]	Data from dir [ack>0]	Data from owner	Inv-Ack	Inv-Ack (last)
I	GetS to dir → IS ^D	GetM to dir → IM ^{AD}										
IS ^D	stall	stall	stall			stall		→ S		→ S		
IM ^{AD}	stall	stall	stall	stall	stall			→ M	→ IM ^A	→ M	ack-- → IM ^{AD}	
IM ^A	stall	stall	stall	stall	stall						ack-- → IM ^A	→ M
S	hit → S	GetM to dir → SM ^{AD}	PutS to dir → SI ^A			Inv-Ack to req → I						
SM ^{AD}	hit → SM ^{AD}	stall	stall	stall	stall	Inv-Ack to req → IM ^{AD}		→ M	→ SM ^A		ack-- → SM ^{AD}	
SM ^A	hit → SM ^A	stall	stall	stall	stall						ack-- → SM ^A	→ M
M	hit → M	hit → M	PutM to dir → MI ^A	Data to req+dir → S	Data to req → I							
MI ^A	stall	stall	stall	Data to req+dir → SI ^A	Data to req → II ^A		→ I					
SI ^A	stall	stall	stall			Inv-Ack to req → II ^A	→ I					
II ^A	stall	stall	stall				→ I					

Enforcing Store Atomicity – Example

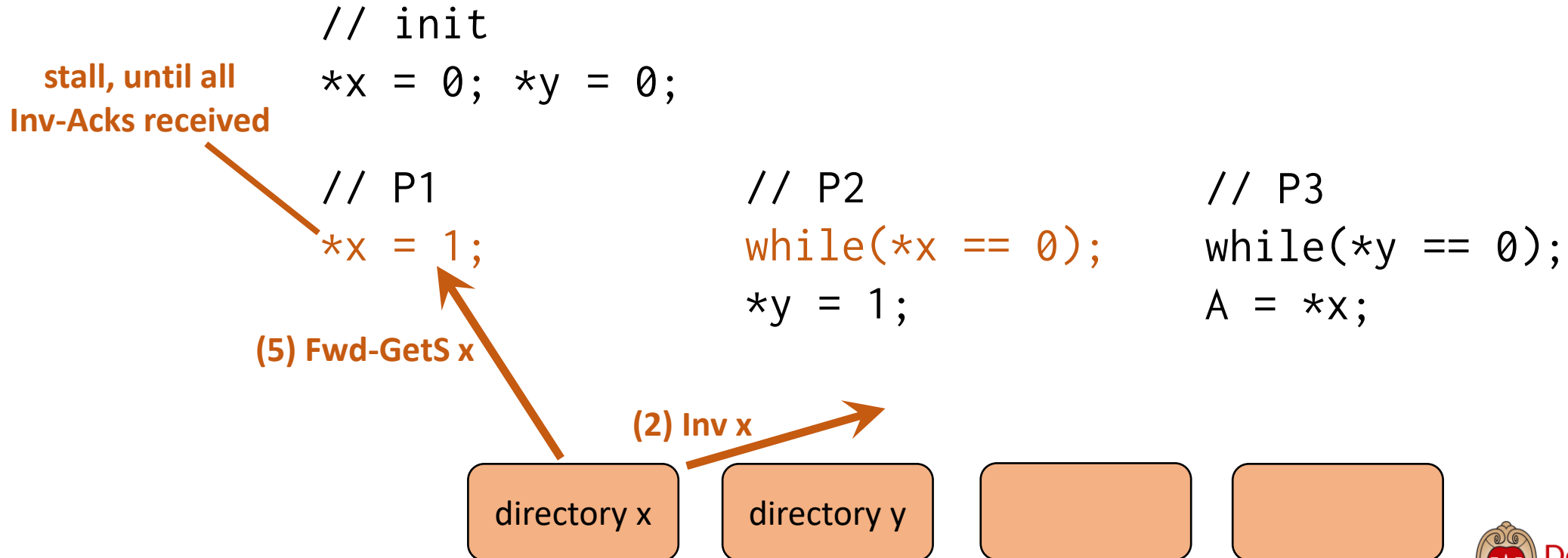
Cache Controller

Current state	Load	Store	Evict	Fwd-GetS	Fwd-GetM	Inv	Put-Ack	Data from dir [ack=0]	Data from dir [ack>0]	Data from owner	Inv-Ack	Inv-Ack (last)
I	GetS to dir → IS ^D	GetM to dir → IM ^{AD}										
IS ^D	stall	stall	stall			stall		→ S		→ S		
IM ^{AD}	stall	stall	stall	stall	stall			→ M	→ IM ^A	→ M	ack-- → IM ^{AD}	
IM ^A	stall	stall	stall	stall	stall						ack-- → IM ^A	→ M
S	hit → S	GetM to dir → SM ^{AD}	PutS to dir → SI ^A			Inv-Ack to req → I						
SM ^{AD}	hit → SM ^{AD}	stall	stall	stall	stall	Inv-Ack to req → IM ^{AD}		→ M	→ SM ^A		ack-- → SM ^{AD}	
SM ^A	hit → SM ^A	stall	stall	stall	stall						ack-- → SM ^A	→ M
M	hit → M	hit → M	PutM to dir → MI ^A	Data to req+dir → S	Data to req → I							
MI ^A	stall	stall	stall	Data to req+dir → SI ^A	Data to req → II ^A		→ I					
SI ^A	stall	stall	stall			Inv-Ack to req → II ^A	→ I					
II ^A	stall	stall	stall				→ I					

Enforcing Store Atomicity – Example

Possible values: A?

(A=1)



Enforcing Store Atomicity – Example

Snooping-Based Coherence:

All stores are atomically globally performed via bus/broadcasts

Update-Based Coherence:

Update messages must be ordered across all nodes

Invalidation-Based Coherence:

Writers must not reveal new values until all invalidations acknowledged

Shared Caches:

Threads that share a cache must not see each other's writes early

Enforcing Store Atomicity – Example

Possible values: A?

(A=1)

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
*x = 1;
```

```
// P2
```

```
while(*x == 0);
```

```
*y = 1;
```

```
// P3
```

```
while(*y == 0);
```

```
A = *x;
```



Enforcing Store Atomicity – Example

Possible values: A?

(A=1)

```
// init
*x = 0; *y = 0;
```

ROB
...
...

```
// P1
*x = 1;
```

```
// P2
while(*x == 0);
*y = 1;
```

```
// P3
while(*y == 0);
A = *x;
```

write-back, write-allocate

L1 cache
[x] = 0
...
...

Enforcing Store Atomicity – Example

Possible values: A?

(A=1)

```
// init
*x = 0; *y = 0;
```

ROB
inst1
*x = 1

```
// P1
*x = 1;
```

```
// P2
while(*x == 0);
*y = 1;
```

```
// P3
while(*y == 0);
A = *x;
```

L1 cache
[x] = 0
...
...

Enforcing Store Atomicity – Example

Possible values: A?

(A=1)

```
// init
*x = 0; *y = 0;
```

ROB
inst2
inst1

```
// P1
*x = 1;
```

```
// P2
while(*x == 0);
*y = 1;
```

```
// P3
while(*y == 0);
A = *x;
```

L1 cache
[x] = 1
...
...



(1) GetM x

Enforcing Store Atomicity – Example

Possible values: A?

(A=1)

```
// init
*x = 0; *y = 0;
```

ROB
inst2
inst1

```
// P1
*x = 1;
```

```
// P2
while(*x == 0);
*y = 1;
```

```
// P3
while(*y == 0);
A = *x;
```

(2) thread migration/context-switch

L1 cache
[x] = 1
...
...

(1) GetM x

Enforcing Store Atomicity – Example

Possible values: A?

(A=1)

```
// init
*x = 0; *y = 0;
```

ROB
inst2
inst1

// P1

while(*x == 0);

*y = 1;

// P2

// P3

while(*y == 0);

A = *x;

(2) thread migration/context-switch

L1 cache
[x] = 1
...
...

(1) GetM x



Enforcing Store Atomicity – Example

Possible values: A?

(A=1)

```
// init
*x = 0; *y = 0;
```

ROB
inst2
inst1

```
// P1
while(*x == 0);
*y = 1;
```

```
// P2
```

```
// P3
while(*y == 0);
A = *x;
```

L1 cache
[x] = 1
...
...



(1) GetM x

Enforcing Store Atomicity – Example

Possible values: A?

(A=1)

```
// init
*x = 0; *y = 0;
```

ROB
inst2
inst1

```
// P1
while(*x == 0);
*y = 1; (3) cache hit x
```

```
// P2
```

```
// P3
while(*y == 0);
A = *x;
```

L1 cache
[x] = 1
...
...



(1) GetM x



Enforcing Store Atomicity – Example

Possible values: A?

(A=1)

```
// init
*x = 0; *y = 0;
```

ROB
inst2
inst1

```
// P1
while(*x == 0);
*y = 1;
```

// P2

```
// P3
while(*y == 0);
A = *x;
```

L1 cache
[x] = 1
...
...

(1) GetM x

Enforcing Store Atomicity – Example

Possible values: A?

(A=1)

```
// init
*x = 0; *y = 0;
```

ROB
inst2
inst1

```
// P1
while(*x == 0);
*y = 1;
```

```
// P2
```

```
// P3
while(*y == 0);
A = *x;
```

L1 cache
[x] = 1
...
...



(1) GetM x

L1 cache
[x] = 0
...
...

(4) cache hit x

Enforcing Store Atomicity – Example

Possible values: A?

(A=1) (A=0)

SC violation!

```
// init
*x = 0; *y = 0;
```

ROB
inst2
inst1

```
// P1
while(*x == 0);
*y = 1;
```

```
// P2
```

```
// P3
while(*y == 0);
A = *x;
```

L1 cache
[x] = 1
...
...

(1) GetM x

L1 cache
[x] = 0
...
...

(4) cache hit x

Enforcing Store Atomicity – Example

Possible values: A?

(A=1)

```
// init
*x = 0; *y = 0;
```

ROB
inst1
*x = 1

```
// P1
*x = 1;
```

```
// P2
while(*x == 0);
*y = 1;
```

```
// P3
while(*y == 0);
A = *x;
```

L1 cache
[x] = 0
...
...

can't retire from ROB and store in
L1 cache until GetM complete

(1) GetM x

Sequential Consistency

Limits latency-hiding optimizations

- e.g., out-of-order execution, compiler techniques, value prediction

Unnecessarily restrictive for parallel programs

- Most multithreaded applications won't notice out-of-order memory accesses

Sequential consistency is what makes sense for programmers...
But enforcing it severely limits optimizations!